

URL になります。これを **ファイルベースルーティング**（File-based Routing：ファイルやフォルダの配置自体がそのままルート定義になる方式）と呼びます。

```
app/                                # この `app/` フォルダが Next.js の出発点
├─ page.tsx                        → /      (トップページ)    # `page.tsx` というファイルがあるフォルダ=そのURLでアクセス可能
├─ about/                          # フォルダ名がURLのセグメント（区切り）になる
│   └─ page.tsx                    → /about    # `app/about/page.tsx` は `/about` のページ本体
└─ books/                          # `/books` 以下のグループ
    ├─ page.tsx                    → /books    # `/books` の一覧ページ
    └─ [id]/                        # 角括弧 `[id]` は「動的セグメント」：数字でも文字でも何でも受ける
        └─ page.tsx                → /books/123 ([id] は動的な値) # 例えば `/books/42` でアクセスすると id="42" として受け取れる
```

ファイル＝ページ: 「`app/about/page.tsx` を作っただけで `/about` というURLでそのページが見られるようになる」というのが Next.js のキモです。第3章までの React だけでは、`react-router-dom` などのライブラリを別途インストールし、自分で `<Route path="/about" element={...} />` のようにルーティング設定を書く必要がありました。Next.js ではその設定作業が「ファイルを置く」という直感的な操作に置き換わります。

0.3 レンダリング3兄弟（CSR / SSR / SSG）

Webページを「画面に出るHTMLにする」処理を **レンダリング**（rendering：データやテンプレートから最終的な見た目のHTMLを組み立てること）と呼びます。これがいつ・どこで行われるかで3種類あります。

種類	フルネーム	いつHTMLができる？	どこで？	メリット
CSR	Client Side Rendering（クライアントサイドレンダリング：ブラウザ側でJSを動かしてHTMLを組み立てる方式）	アクセス時	ブラウザ	動的、軽量サーバー
SSR	Server Side Rendering（サーバーサイドレンダリング：サーバー側で完成形HTMLを作って送る方式）	アクセス時	サーバー	SEO良、初回表示速い
SSG	Static Site Generation（スタティックサイトジェネレーション：ビルド時にHTMLを事前生成しておく方式）	ビルド時に1回	サーバー	最速、安価

このほか **ISR**（Incremental Static Regeneration：インクリメンタル・スタティック・リジェネレーション。SSGをベースに「指定秒ごとに再生成」する仕組み）、**RSC**（React Server Components：リアクト・サーバー・コンポーネンツ。サーバーでだけ動く新しい種類のReactコンポーネント）、**ストリーミング**（Streaming：完成した部分から順次ブラウザに送る方式）、**ハイドレーション**（Hydration：サーバーで出来上がったHTMLにブラウザ側のJSが「水を与える」ように後付けで動きを付ける処理）という言葉もよく出てくるので、本章で順番に扱います。

本書での使い分け: Next.js の App Router では、`page.tsx` をシンプルに書くとサーバーで実行される（SSR/SSG相当）のがデフォルトです。インタラクティブな部品（ボタンや入力欄など）だけ `"use client"` を付けて Client Component（クライアントコンポーネント：ブラウザ側で動くコンポーネント）にし、CSR的な動きを実現します。

はじめに

この章では、React ベースのフルスタックフレームワーク（Full-stack Framework：フロントエンド＝画面もバックエンド＝サーバー処理も両方カバーするフレームワーク）である **Next.js**（ネクストジェイエス）の基礎を学びます。

Next.js を使うことで、React 単体では実現が難しい**サーバーサイドレンダリング**（SSR：Server Side Rendering = サーバー側でHTMLを生成してからブラウザに送る方式。ページの初回表示が速くなる）、**静的サイト生成**（SSG：Static Site Generation = ビルド時にHTMLを事前生成する方式。最も高速）、**ファイルベースルーティング**（ファイルを配置するだけでURLとページが自動的に対応する仕組み）などを簡単に実装できます。

なぜ React だけではなく Next.js を使うのか

React は「UIの部品を作るライブラリ」であり、それ以外の機能（ページのURL管理、サーバーとの通信、SEO対策など）は自分で用意する必要があります。Next.js はこれらを**最初から備えている**ため、開発者はアプリのロジックに集中できます。

機能	React のみ	Next.js
ページのURL管理（ルーティング）	別途ライブラリ（react-router等）が必要	ファイルを置くだけで自動対応
サーバーでのデータ取得	自分でAPIサーバーを構築	Server Components で直接取得
SEO（検索エンジン対策）	追加設定が必要	標準で対応
ページ読み込み速度	初回が遅い（CSR）	高速（SSR/SSG）
デプロイ（公開）	別途設定が必要	Vercel で数クリック

この章で学ぶこと: - Next.js の概念と React との関係 - **App Router**（アップルーター：Next.js 13以降の新しいルーティング方式）によるルーティング - **Server Components**（サーバー上で実行されるコンポーネント）と **Client Components**（ブラウザ上で実行されるコンポーネント）の使い分け - レイアウト、ナビゲーション、**データフェッチ**（Data Fetch：サーバーやAPIからデータを取得すること）の基本 - **Server Actions**（サーバーアクション：フォーム送信などのサーバー処理をシンプルに書ける仕組み）によるサーバー処理 - プロジェクト構成の**ベストプラクティス**（Best Practice：最も良いとされるやり方・慣習）

1. Next.js とは

1.1 React との関係

Next.js は **Vercel 社** が開発・メンテナンスしている React ベースのフレームワークです。React はあくまで「UIライブラリ」であり、ルーティングやサーバーサイド処理などは含まれていません。Next.js はその React の上に、Web アプリケーション開発に必要な機能を包括的に提供します。



たとえ話で理解する:

- **React** = エンジン（車を動かす核心部分）
- **Next.js** = 完成した車（エンジンに加え、ハンドル、ブレーキ、ナビなど全部入り）

React 単体でアプリを作ることできますが、ルーティングやデータ取得の仕組みを自分で選定・構築する必要があります。Next.js を使えば、それらが最初から統合されています。

1.2 SSR, SSG, CSR の違い

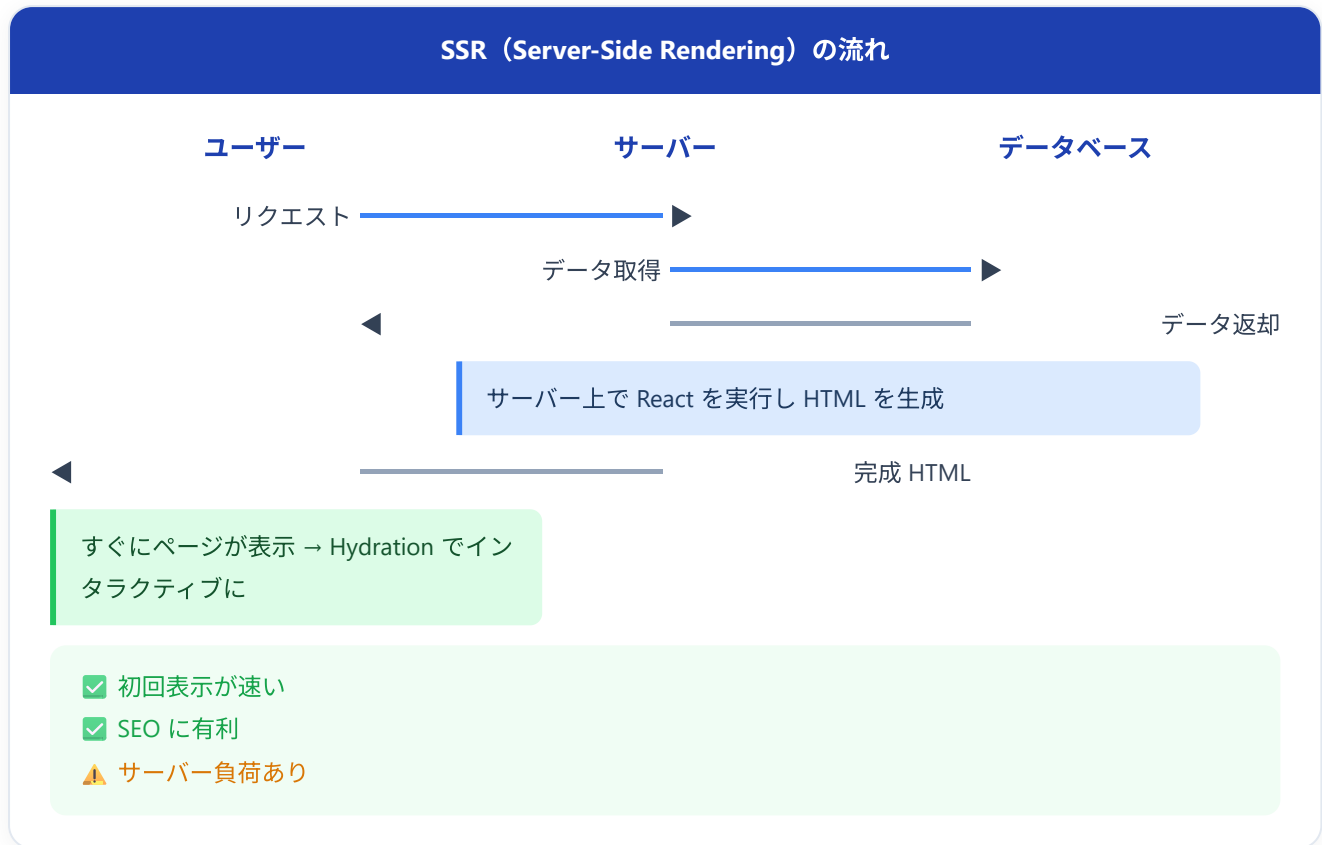
Web ページのレンダリング方法は大きく3つあります。Next.js はこれらすべてに対応しています。

CSR (Client-Side Rendering) - クライアントサイドレンダリング



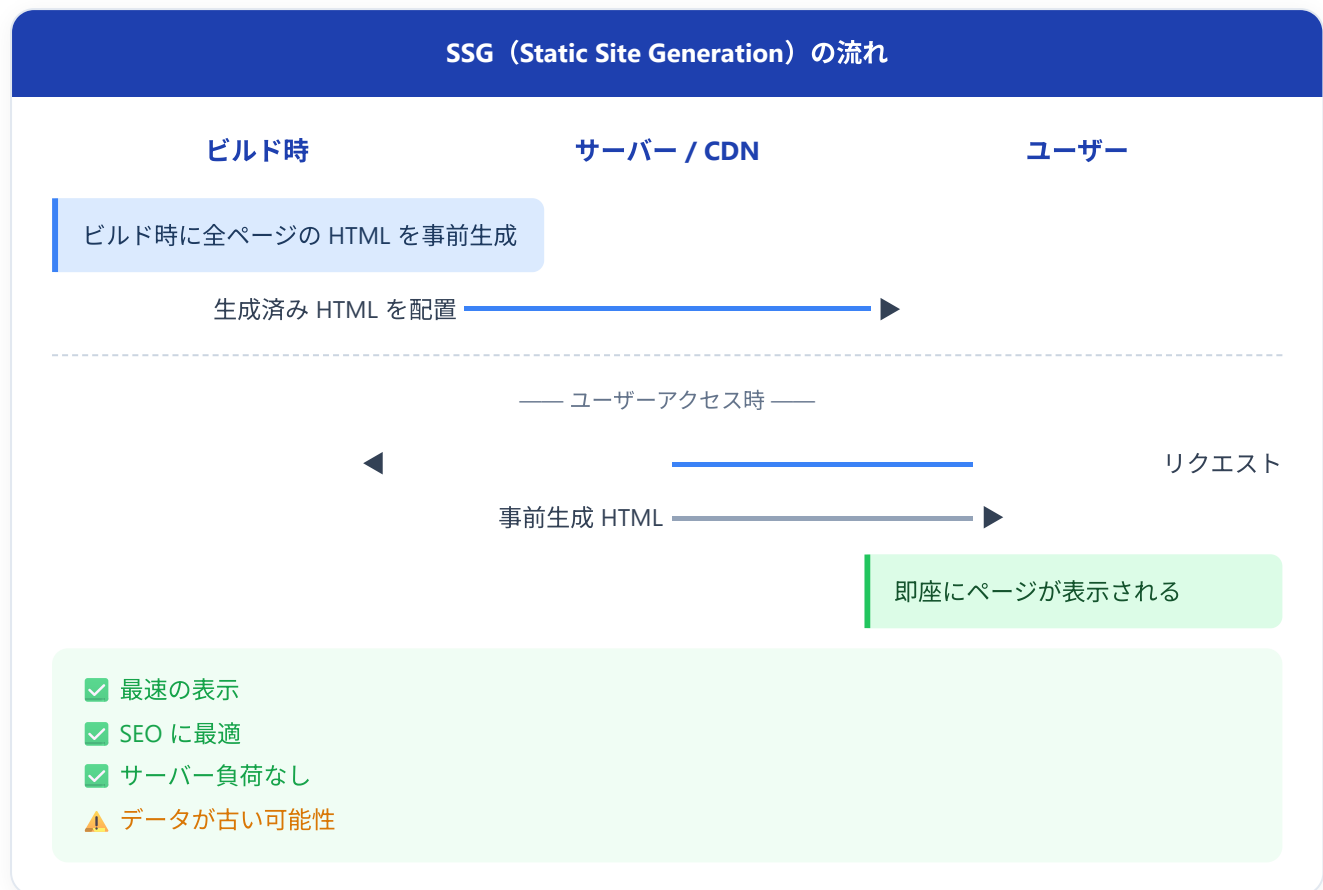
画面にはこう表示される: 最初に一瞬だけ白い画面やローディングスピナーが表示され、JavaScript の処理が完了してからコンテンツが表示されます。素の React (Create React App) はデフォルトでこの方式です。

SSR (Server-Side Rendering) - サーバーサイドレンダリング



画面にはこう表示される: リクエストのたびにサーバーで HTML が生成されるため、最新のデータが反映された完成形のページがすぐに表示されます。ブラウザが JavaScript を読み込むと、ボタンクリックなどのインタラクションが可能になります（この過程を **Hydration** と呼びます）。

SSG（Static Site Generation） - 静的サイト生成



画面にはこう表示される: ビルド時に生成済みの HTML がそのまま返されるため、表示速度は最速です。ただし、ビルド後にデータが変わっても、再ビルドするまで古いデータが表示され続けます。ブログや企業サイトなど、更新頻度の低いページに最適です。

3つのレンダリング方式の比較

特性	CSR	SSR	SSG
初回表示速度	遅い	速い	最速
SEO	不利	有利	最も有利
データの鮮度	常に最新	常に最新	ビルド時点
サーバー負荷	低い	高い	最も低い
使用例	ダッシュボード	EC サイト商品ページ	ブログ記事

1.3 なぜ Next.js を使うのか（素の React との比較）

機能	素の React	Next.js
ルーティング	react-router 等を別途インストール	App Router が組み込み
SSR / SSG	自分で構築が必要（非常に複雑）	設定なしで利用可能
コード分割	手動で React.lazy 等を使う	自動で最適化
画像最適化	自分で実装	<code>next/image</code> が自動最適化
フォント最適化	自分で実装	<code>next/font</code> が自動最適化
API エンドポイント	Express 等の別サーバーが必要	Route Handlers / Server Actions
TypeScript	設定が必要	初期設定済み
ESLint	設定が必要	初期設定済み
環境変数	dotenv 等を別途設定	<code>.env.local</code> が組み込み
デプロイ	ホスティング先の選定・設定が必要	Vercel にワンクリックデプロイ

結論: 2024年以降、新規の React プロジェクトを始める場合は Next.js（App Router）を使うのがスタンダードです。React の公式ドキュメントでも Next.js が推奨フレームワークの1つとして紹介されています。

2. App Router

2.1 App Router vs Pages Router

Next.js には2つのルーティングシステムがあります。

特性	Pages Router (従来)	App Router (推奨)
ディレクトリ	<code>pages/</code>	<code>app/</code>
導入バージョン	Next.js 初期～	Next.js 13.4～ (安定版)
デフォルトコンポーネント	Client Component	Server Component
レイアウト	<code>_app.tsx</code> , <code>_document.tsx</code>	<code>layout.tsx</code> (ネスト可能)
データ取得	<code>getServerSideProps</code> , <code>getStaticProps</code>	async/await で直接
ローディング UI	自分で実装	<code>loading.tsx</code>
エラー UI	<code>_error.tsx</code> (グローバル)	<code>error.tsx</code> (ルートごと)
Server Actions	非対応	対応
Streaming	非対応	対応

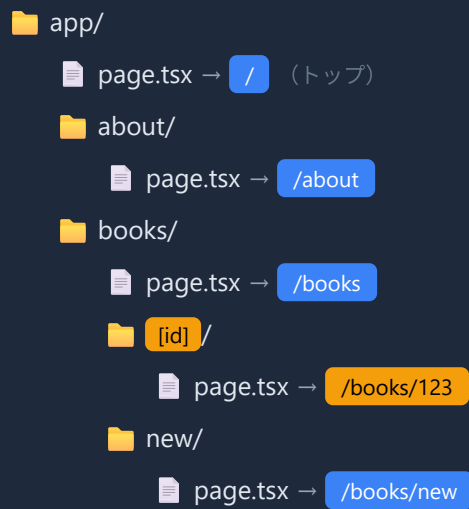
本チュートリアルでは **App Router** のみを使用します。Pages Router (ページズルーター: `pages/` フォルダを使う Next.js の初期からあるルーティング方式) は旧バージョンとの互換性のために残されていますが、新規プロジェクトでは App Router を選択してください。Pages Router では「コンポーネントは原則 Client、データ取得は `getServerSideProps` などの専用関数で外側から差し込む」という設計でしたが、App Router では「コンポーネント自体を async にして直接 await する」という根本的に新しい設計になっています。

なぜデフォルトが Server Component なのか: App Router は React Server Components (RSC: サーバーで実行されブラウザに HTML だけが送られる新種のコンポーネント) を基盤にしています。ブラウザに送る JS の量を減らし、機密情報の漏えいを防ぎ、初回表示を速くするために「まずは全部サーバー側で動かす」のがデフォルトなのです。クライアントでの動きが必要な箇所だけを `"use client"` で明示的に切り出していきます。

2.2 ファイルベースルーティングの仕組み

Next.js の App Router では、`app/` ディレクトリ内のフォルダ構造がそのまま URL のパスに対応します。これが **ファイルベースルーティング** です。

ファイル構造 → URL パス



重要なルール: - フォルダ がURLのセグメント（パスの一部）になる - `page.tsx` があるフォルダだけがアクセス可能なルートになる - `page.tsx` がないフォルダは URL として公開されない（コンポーネントの整理用に使える）

2.3 page.tsx, layout.tsx, loading.tsx, error.tsx の役割

App Router では、特別な名前を持つファイルにそれぞれ役割があります。

app/books/ ディレクトリ

layout.tsx (共通レイアウト)

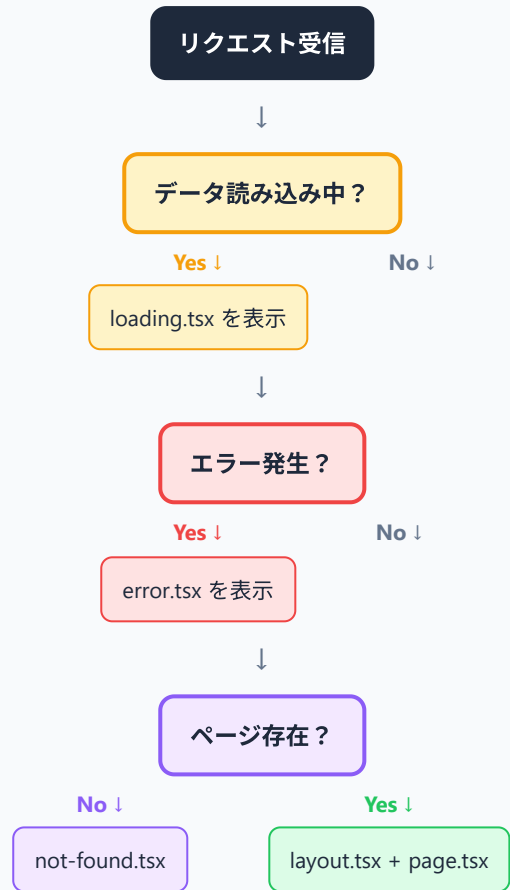
page.tsx (ページ本体)

loading.tsx (読み込み中の表示)

error.tsx (エラー時の表示)

not-found.tsx (404の表示)

レンダリングの流れ



page.tsx - ページ本体

page.tsx (ページ・ティーエスエックス) は Next.js が「これがこのフォルダのページ本体」と認識する予約名のファイルです。そのルート (URL) にアクセスしたときに表示されるメインコンテンツを書きます。**page.tsx** がないフォルダは URL として公開されない、というルールも合わせて覚えておきましょう。

▼このコードがやること (先に日本語で) : **app/page.tsx** を作って「トップページ (URL は /) の中身」を定義します。コンポーネント (画面の部品) は「JSX を返す関数」で、**export default** を付けて「このファイルの主演」として外に公開するのがポイントです。見出しと段落を表示するだけのシンプルなページです。詳しい1行ずつの意味はコード内コメントを見てください。

```

// app/page.tsx
// (ルート) になる
// URL: /

export default function HomePage() {
  // `export default` でこのコンポーネントを「このファイルの主役」として外に公開

  // 関数名 `HomePage` は何でもよい（慣習的にページ名を付ける）。Next.js は名前ではなく default export を見ている
  return (
    <div>
      // JSX (HTMLに似たReactの記法) を返す
      // `<div>` : レイアウト用の汎用ブロック要素。中身を1つにまとめるために使う
      <h1>書籍管理アプリへようこそ</h1>
      // `<h1>` : ページの一番大きな見出し。1ページに1つだけ置くのが推奨
      <p>あなたの読書記録を管理しましょう。</p>
      // `<p>` : 段落 (paragraph) 。普通の記事ブロック
    </div>
    // 閉じタグ。開いた `<div>` と必ず対応させる
  );
  // return の終わり
  // 関数の終わり
}

```

画面にはこう表示される: ブラウザで `http://localhost:3000/` にアクセスすると、「書籍管理アプリへようこそ」という見出しと「あなたの読書記録を管理しましょう。」という段落が表示されます。

補足: なぜデフォルトが Server Component なのか? App Router では `page.tsx` の中身はサーバーで実行され、結果のHTMLだけがブラウザに送られます。これは「JavaScript バンドル（ブラウザに送られるJSの塊）を小さく保つ」「DB接続情報など秘密の値をブラウザに漏らさない」「初回表示を速くする」という3つの利点があるからです。ボタンクリックなどユーザー操作が必要になったときだけ `"use client"` で Client Component に切り替えます。

layout.tsx - 共通レイアウト

複数のページで共有される UI（ヘッダー、サイドバーなど）を定義します。ページ遷移してもレイアウトは再レンダリングされず、状態が保持されます。

▼このコードがやること（先に日本語で）: すべてのページを「共通ヘッダー + 本文 + フッター」という枠で包む、アプリの一番外側の土台を作ります。 `children`（チルドレン）という名前の引数に「各ページの中身」が自動で差し込まれる仕組みがキモで、ページを移動してもこの枠は作り直されません。 `<html>` と `<body>` タグはこのルートレイアウトにだけ書く、という決まりも押さえてください。

```
// =====
// ファイルパス: app/layout.tsx
// 役割      : すべてのページを「ヘッダー+本文+フッター」の枠で包む
//            = Next.jsアプリの **一番外側の共通レイアウト**
// -----
// このファイルは Next.js が自動的に認識する「予約名」のひとつ。
// app/ フォルダ直下に layout.tsx が必ず1つ必要。
// =====

// `Metadata` 型を import する (Next.js 標準の型)。
// `import type` は「型だけ取り込む」記法。実行時のJSには残らないので軽量。
import type { Metadata } from "next";

// -----
// (1) メタデータの定義
// -----
// metadata という名前で export すると、Next.js が <head> 内の <title> や
// <meta name="description" ...> を自動生成してくれる。
// 自分で <head> を書く必要がない。
export const metadata: Metadata = {
  title: "書籍管理アプリ", // ブラウザタブの文字
  description: "あなたの読書記録を管理するアプリケーション", // SEO・SNS共有時の説明文
};

// -----
// (2) ルートレイアウトのコンポーネント本体
// -----
// `RootLayout` は予約名ではないが、慣習的にこの名前を付ける。
//
// 引数 (Props) :
//   - children: このレイアウトの中に差し込まれる「各ページの中身」
//   React.ReactNode 型は「JSX/文字列/数値/null/...などReactで扱える何でも」を表す。
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    // — 必須: <html> と <body> はルートレイアウトでだけ書く —
    // lang="ja" は「このページは日本語ですよ」という指定。
    // スクリーンリーダーや翻訳ツールがこれを参照する。
    <html lang="ja">
      <body>

        {/* — 共通ヘッダー (全ページに表示される) — */}
        <header>
          <nav>
            <h1>  書籍管理アプリ</h1>
          </nav>

```

```

</header>

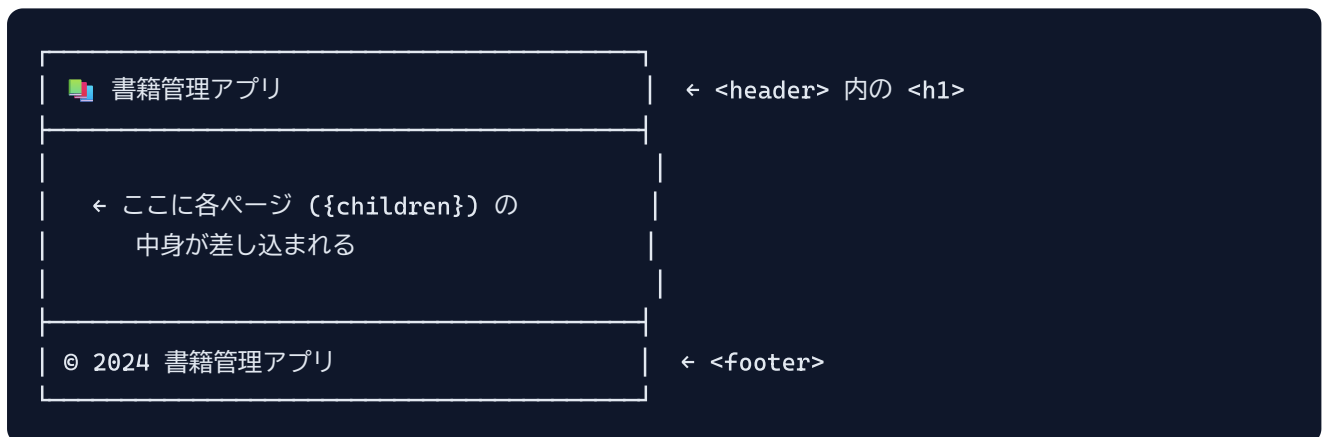
{/*
  — ページ本体が差し込まれる場所 —
  各 page.tsx の return 値が、ここに自動で入る。
  /books にアクセスしたら books/page.tsx の内容が、
  /books/123 にアクセスしたら books/[id]/page.tsx の内容がここに入る。
*/}
<main>{children}</main>

{/* — 共通フッター（全ページに表示される） — */}
<footer>
  <p>&copy; 2024 書籍管理アプリ</p>
</footer>

</body>
</html>
);
}

```

▼ ブラウザでの見た目（どのページでも上下にこの枠が出る）：



▼ 動作の仕組み:

URL	layout.tsx	中央 ({children})
/	表示される	app/page.tsx の中身
/books	表示される	app/books/page.tsx の中身
/books/123	表示される	app/books/[id]/page.tsx の中身

ページ遷移してもヘッダーとフッターは再描画されない（=スクロール位置や状態が保たれる）のが Next.js のレイアウト機能のメリットです。

loading.tsx - 読み込み中の表示

`loading.tsx` は Next.js が「データ取得中に表示するUI」と認識する予約名のファイルです。データの読み込み中に自動的に表示される UI を書きます。仕組みとしては内部で React の `<Suspense>`（サスペンス：データ待ちの間に代わりのUIを出す機能）を使っており、対応する `page.tsx` の Server Component が `await` でデータを待っている間、この `loading.tsx` の中身が代わりに描画されます。

▼ このコードがやること（先に日本語で）：データ取得中に「読み込み中」を伝える画面（ローディングUI）を作ります。`loading.tsx` という予約名のファイルを置くだけで、Next.js が自動で「ページの準備ができるまでの間」にこれを表示してくれます。回転するスピナーと案内メッセージを出すだけのシンプルな部品です。詳細はコード内コメントを参照してください。

```
// app/books/loading.tsx                                     // ファイルパス。`/books` のロード中UIになる
                                                             // フォルダごとに置けるので、ページ単位で異なるローディング表現が可能

export default function BooksLoading() {                     // default export された関数がそのまま「ローディングUI」として使われる
                                                             // 関数名は何でもよいが、慣習的に
  `XxxLoading` と付ける
  return (                                                    // 返す JSX
    <div className="loading-container">                       // `className` は HTML の `class` 属性の
      JSX版（`class` はJSの予約語なので別名）
      <div className="spinner" />                             // CSSアニメーションで作る回転するスピナー（自
      己終了タグ `</>` で書ける）
      <p>書籍データを読み込んでいます...</p>                 // ユーザーへの説明文
    </div>                                                     // ラッパーdivの閉じタグ
  );
}                                                             // 関数の終わり
```

画面にはこう表示される: `/books` にアクセスした直後、データベースからデータを取得している間、スピナーアニメーションと「書籍データを読み込んでいます...」というメッセージが表示されます。データの取得が完了すると、自動的に `page.tsx` の内容に切り替わります。これは内部的に「サーバーがHTMLを少しずつ送る＝ストリーミング」と呼ばれる挙動で実現されています。

error.tsx - エラー時の表示

`error.tsx` は Next.js が「そのフォルダ内でエラーが起きたときに表示するUI」と認識する予約名のファイルです。ページの描画中に例外が `throw` された場合などに表示される UI を書きます。`"use client"` が必須です（React の Error Boundary（エラーバウンダリ：エラーをキャッチして代替UIを出す仕組み）はクラスコンポーネントを内部で使うため、Client Component でないと動かないため）。

▼このコードがやること（先に日本語で）：ページ表示中にエラー（不具合）が起きたとき、白い画面で固まる代わりに「エラーが発生しました」という案内と「もう一度試す」ボタンを出す画面を作ります。`error.tsx` には必ず先頭に `"use client"` を書く必要があり、Next.js が `error`（起きたエラー情報）と `reset`（再挑戦用の関数）を自動で渡してくれます。ボタンを押すと `reset()` が呼ばれてその部分の再表示を試みます。詳細はコード内コメントを参照してください。

```
// app/books/error.tsx
表示される
"use client";
コンポーネント」に切り替える

// `books` 配下でエラーが起きたときに自動で
// この1行でこのファイルを「ブラウザ側で動く
// error.tsx ではこれを必ず先頭に書く（ファ
イルの一番上、importより前）

export default function BooksError({
  呼び出す
  error,
  ト
  reset,
  Next.js が渡してくれる関数
}): {
  error: Error & { digest?: string };
  `digest`（任意）を合成した型

  // default export 関数。Next.js が自動で
  // props.error: 発生した Error オブジェク
  // props.reset: 「もう一度試す」用に

  // ↓ ここから TypeScript の型注釈
  // 標準 Error 型に、Next.js が独自に付ける
  // digest はサーバーログとの紐付け用ID（本番
  環境で詳細メッセージを隠したいとき使う）
  reset: () => void;
}) {
  return (
    <div className="error-container">
      <h2>エラーが発生しました</h2>
      <p>{error.message}</p>
      <button onClick={() => reset()}>もう一度試す</button>
    </div>
  );
}
```

画面にはこう表示される: 書籍データの取得中にエラーが発生すると、「エラーが発生しました」という見出しとエラーメッセージ、そして「もう一度試す」ボタンが表示されます。ボタンを押すと、そのセグメントの再レンダリングが試みられます。

補足: `not-found.tsx` もある: `notFound()` 関数（後述）が呼ばれたときや404相当の状態のときに表示するUIとして `not-found.tsx` という予約ファイルも置けます。`error.tsx` と兄弟関係にあります。

2.4 動的ルーティング ([id])

URL の一部をパラメータ（parameter：動的に変わる値）として受け取りたい場合、フォルダ名を角括弧 `[...]` で囲みます。これを **動的セグメント**（Dynamic Segment：URLの中で変化する部分）と呼びます。

```
app/                                # アプリのルートフォルダ
  books/                            # `/books` 以下のグループ
    [id]/                          # 角括弧で囲むと「ここは可変」を意味する。フォルダ名の `id`
    が変数名になる
      page.tsx    → /books/1, /books/2, /books/abc などにマッチ  # 数字でも文字列でも何でも
    受ける
```

▼ このコードがやること（先に日本語で）：URL の一部（例: `/books/42` の「42」）をプログラムの中で受け取って画面に表示します。フォルダ名を `[id]` のように角括弧で囲むと、その部分が変化する値（パラメータ）になり、`params` という引数で受け取れます。Next.js 15 以降では `params` は `await` で取り出す必要がある点に注意してください（受け取った値は数値ではなく文字列で来ます）。詳細はコード内コメントを参照してください。

```

// app/books/[id]/page.tsx
動的セグメント

// params はオブジェクトとして渡される
// Next.js 15 では params は Promise として渡されるため await が必要
export default async function BookDetailPage({
  // `async` 関数にすると本体で `await`
  // この `async function` をコンポー
  // props.params: 動的セグメントの値
  // ↓ TypeScript 型注釈
  // Next.js 15+ では params は
  // `{ id: string }` は `id` という文
  params: Promise<{ id: string }>;
  Promise でラップされて渡される

  字列プロパティを持つオブジェクト」型
}) {
  const { id } = await params;
  割代入で `id` だけ取り出す

  "42" (数値ではなく文字列!)

  return (
    <div>
      <h1>書籍詳細</h1>
      <p>書籍 ID: {id}</p>
    </div>
  );
}

// ページ全体のラッパー
// 見出し
// 取り出した id を JSX に埋め込ん
/* 実際にはここで id を使ってデータベースから書籍情報を取得する */
// JSX の中のコメントは `{/* ...`
// 例: const book = await
getBookById(id);

```

画面にはこう表示される: - `/books/1` にアクセスすると「書籍 ID: 1」と表示されます。 - `/books/42` にアクセスすると「書籍 ID: 42」と表示されます。 - `/books/abc` にアクセスすると「書籍 ID: abc」と表示されます。

URL のその部分がそのまま `id` パラメータとして使えるわけです。

複数の動的セグメント

```
app/  
  users/  
    [userId]/  
      books/  
        [bookId]/  
page.tsx → /users/5/books/12 にマッチ
```

▼ このコードがやること（先に日本語で）：URL に変化する部分が2つある場合（例: `/users/5/books/12`）に、その両方の値（`userId` と `bookId`）を同時に取り出して表示します。動的セグメントは2段以上ネストで、`params` の中に複数のキーとして入ってきます。`await` で取り出してから分割代入で2つの変数に展開するのがポイントです。詳細はコード内コメントを参照してください。

```
// =====
// ファイルパス: app/users/[userId]/books/[bookId]/page.tsx
// 役割      : URL 中の userId と bookId を取り出して表示するページ
// -----
// このファイルは Next.js が「動的セグメントを含むページ」と認識する。
// /users/5/books/12 にアクセスされると userId="5", bookId="12" になる。
// =====

// (1) 関数本体に async を付けると、内部で await が使える。
//     コンポーネント関数を async にできるのは Server Component だけの特権。
//
// (2) 引数の Props 型は { params: Promise<{ ... }> }
//     Next.js 15 以降では params が Promise でラップされて渡される。
//     なので await で「中身の値」を取り出してから使う。
export default async function UserBookPage({
  params,
}: {
  params: Promise<{ userId: string; bookId: string }>;
}) {
  // (3) Promise を await してオブジェクトを取り出し、
  //     さらに分割代入で userId と bookId に展開する。
  //     URL が /users/5/books/12 なら userId = "5", bookId = "12" となる。
  //     ※ 文字列で来ることに注意。数値が必要ななら Number(userId) で変換する。
  const { userId, bookId } = await params;

  // (4) 取り出した値を JSX 内に埋め込んで表示
  return (
    <div>
      <p>ユーザー ID: {userId}</p>
      <p>書籍 ID: {bookId}</p>
    </div>
  );
}

// ▼ /users/5/books/12 にアクセスしたときの画面表示:
//   ユーザー ID: 5
//   書籍 ID: 12
```

キャッチオールセグメント

「何階層あっても全部受け取りたい」という場合は `[...slug]` のように `...`（スプレッド構文と同じ記号）を付けます。これを **Catch-all Segment**（キャッチオールセグメント：何階層でも一括で受け取る動的セグメント）と呼びます。

```

app/                # ルート
  docs/             # `/docs` 以下のグループ
    [...slug]/      # `...` 付きの動的セグメント。slug は配列
で渡る
  page.tsx          → /docs/a, /docs/a/b, /docs/a/b/c などにマッチ # 何階層でもOK

```

▼ このコードがやること（先に日本語で）： `/docs/a/b/c` のように何階層続くか分からない URL を、まとめて1つのページで受け取ります。フォルダ名を `[...slug]` のように `...` 付きにすると、それ以降のパスが「文字列の配列」として渡ってきます（例： `["react", "hooks", "useEffect"]` ）。受け取った配列を `join(" / ")` でつなげて表示する例です。詳細はコード内コメントを参照してください。

```

// =====
// ファイルパス: app/docs/[...slug]/page.tsx
// 役割      : スラッシュ区切りで何階層でもマッチする「キャッチオール」ページ
// =====
// [...slug] のように ... を付けると、それ以降のすべてのパスが配列として渡る。
// /docs/react          → slug = ["react"]
// /docs/react/hooks     → slug = ["react", "hooks"]
// /docs/react/hooks/useEffect → slug = ["react", "hooks", "useEffect"]
// =====

export default async function DocsPage({
  params,
}: {
  // (1) slug は文字列の配列として渡る
  params: Promise<{ slug: string[] }>;
}) {
  // (2) Promise から slug 配列を取り出す
  const { slug } = await params;
  // 例: /docs/react/hooks/useEffect → slug = ["react", "hooks", "useEffect"]

  // (3) Array.prototype.join(" / ") で配列を文字列にする
  // ["react", "hooks", "useEffect"].join(" / ") → "react / hooks / useEffect"
  return (
    <div>
      <p>パス: {slug.join(" / ")}</p>
    </div>
  );
}

// ▼ /docs/react/hooks/useEffect にアクセスしたときの画面表示:
//   パス: react / hooks / useEffect

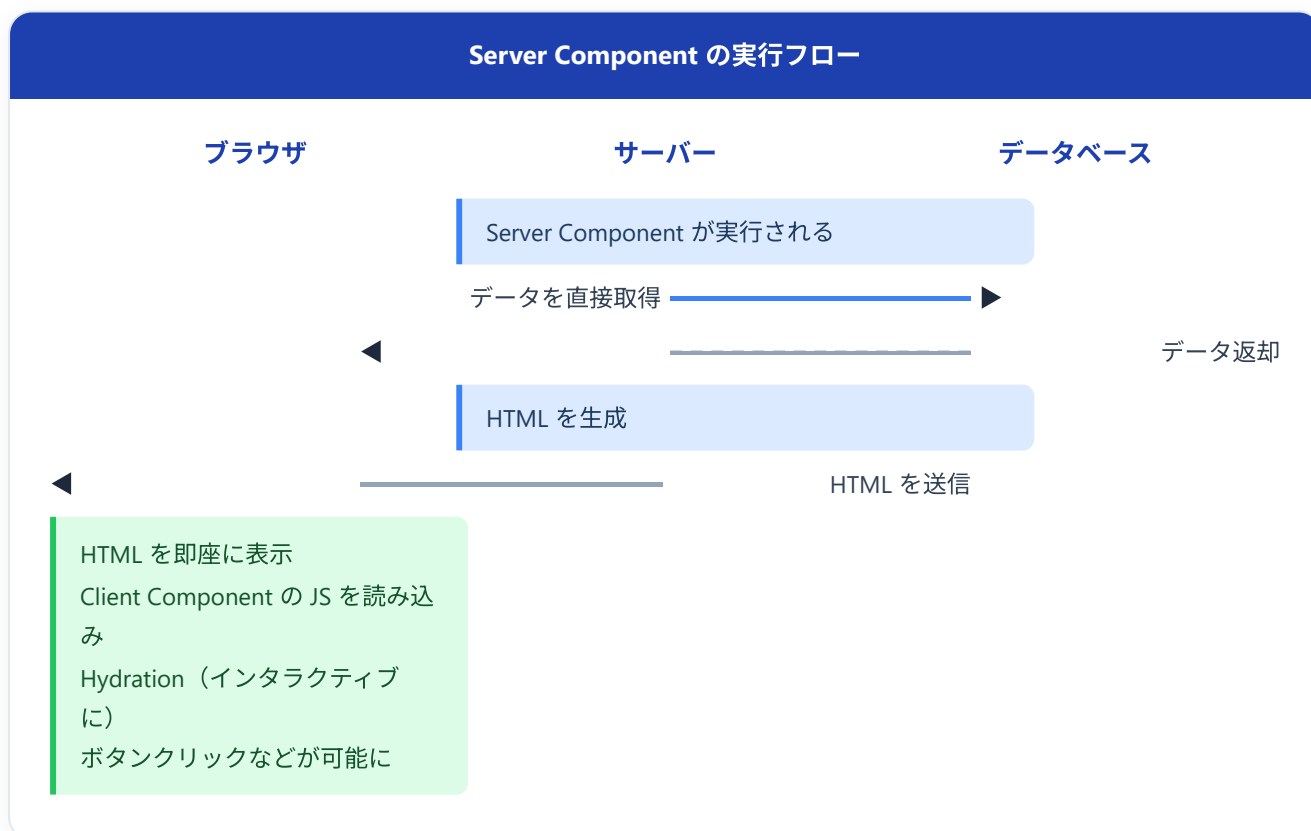
```

3. Server Components vs Client Components

3.1 概念の説明

Next.js App Router の最大の特徴は、デフォルトですべてのコンポーネントが **Server Component** であるということです。

Server Components (デフォルト)	Client Components ('use client')
✓ サーバー上で実行される ✓ データベースに直接アクセス可能 ✓ JS バンドルに含まれない ✗ <code>useState</code>, <code>useEffect</code> 使用不可 ✗ <code>onClick</code> 等のイベント処理不可 ✗ ブラウザ API 使用不可	🌐 ブラウザ上で実行される 🌐 API 経由でデータ取得 🌐 JS バンドルに含まれる ✓ <code>useState</code>, <code>useEffect</code> 使用可能 ✓ <code>onClick</code> 等のイベント処理可能 ✓ ブラウザ API 使用可能



3.2 "use client" ディレクティブ

Client Component にするには、ファイルの **先頭**（import 文よりも上）に **"use client"** と記述します。これを **ディレクティブ**（directive：コンパイラやランタイムに対する命令文）と呼びます。

'use client' の伝播ルール（重要）：- **"use client"** を書いたファイルは、その時点で「Client Component の境界（ボーダー）」になります。- その境界より下流（そのファイルが import する他のファイル）は、明示しなくても自動的に Client Component として扱われます。- つまり「Client から import される Server Component」は実質作れません。- 逆に「Server Component から import される Client Component」はOKです。これが普通の組み合わせ。- 1つのページ内で「Server の骨組みの中に Client 部品を埋め込む」という構造が App Router の基本パターンです。

▼このコードがやること（先に日本語で）：「**"use client"** を書かないと自動的に Server Component（サーバー側で動く部品）になる」というデフォルトの形を確認します。Server Component は関数を **async** にしてサーバー上で直接データを取りに行ける反面、**useState** やボタンのクリック処理などブラウザ側の機能は使えません。ここではデータ取得は省略し、文字を表示するだけの最小例です。詳細はコード内コメントを参照してください。

```
// =====  
// Server Component の例 ("use client" を書かない=デフォルト)  
// =====  
// このファイルはサーバーでだけ実行される。  
//   ✓ 関数を async にして await でDBやAPIにアクセスできる  
//   ✓ 機密情報（APIキーなど）を扱う（ブラウザに送られない）  
//   ✗ useState / useEffect / onClick などのインタラクティブ機能は使えない  
//   ✗ window / document などブラウザ専用のAPIは使えない  
  
export default function BookList() {  
  // (1) 通常はここで await fetch(...) や DBクライアントを呼んでデータを取得する  
  //   例: const books = await db.query("SELECT * FROM books");  
  return <div>書籍一覧（サーバーで描画）</div>;  
}
```

▼このコードがやること（先に日本語で）：ファイル先頭に **"use client"** を書いて、ブラウザ側で動く Client Component（クライアントコンポーネント）にする例です。これにより **useState**（状態を覚えておく仕組み）やユーザー入力への反応が使えるようになります。入力欄に文字を打つたびに状態が更新され、画面の表示もそれに追従する「検索バー」を作ります。詳細はコード内コメントを参照してください。

```
// =====
// Client Component の例 ("use client" をファイル先頭に書く)
// =====
// このディレクティブが書かれた瞬間から、このファイルとそこから import される
// すべてのコンポーネントはブラウザで動く扱いになる。
"use client";

// (1) Client Component ではブラウザ側で動く React のフックが使える
import { useState } from "react";

export default function SearchBar() {
  // (2) useState で「検索クエリ文字列」を状態として管理する
  //   [現在の値, 値を変える関数] = useState(初期値)
  //   初期値が "" なので最初の query は空文字。
  const [query, setQuery] = useState("");

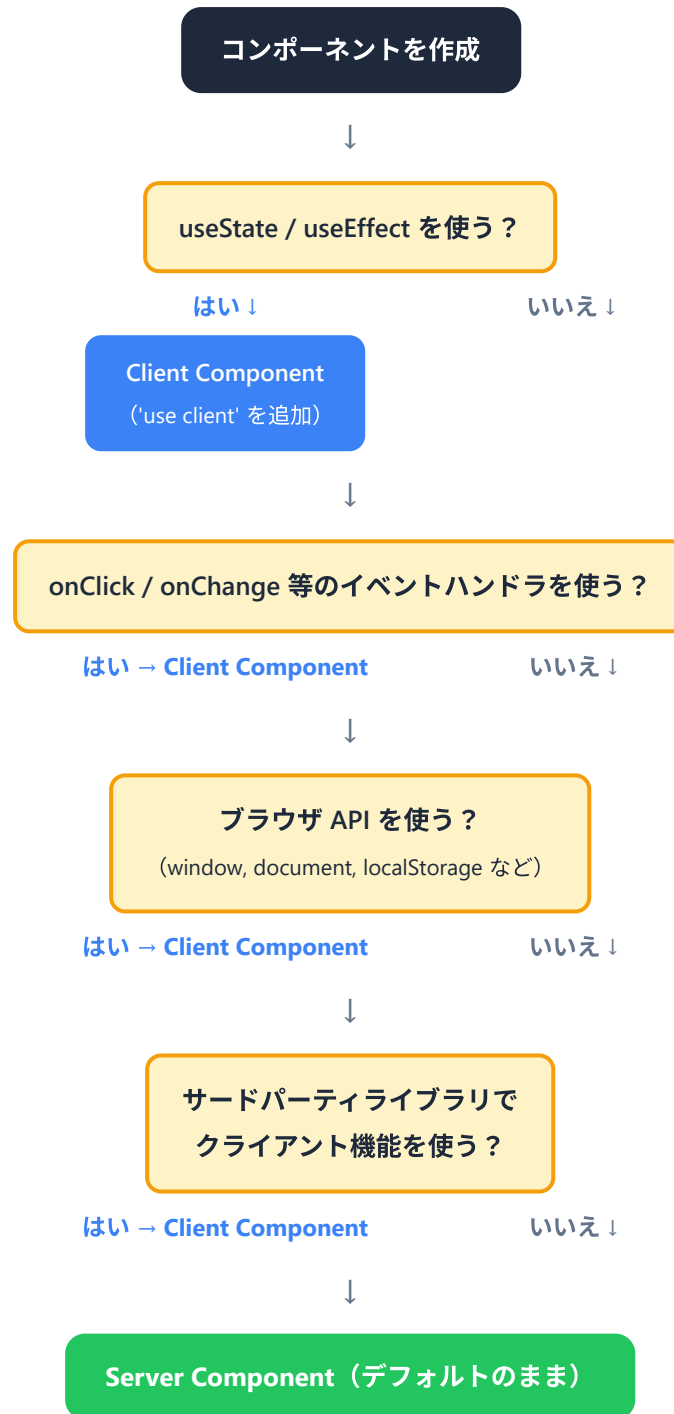
  // (3) JSX で <input> を描画
  //   ・value={query}      : 入力欄に現在の状態を反映する (制御コンポーネント)
  //   ・onChange={e=>...}: ユーザーが文字を打つたびに呼ばれる関数
  //   ・e.target.value     : 入力欄の現在の値 (input要素のテキスト)
  //   ・setQuery(...)     : 状態を更新 → React が再描画 → value も更新される
  return (
    <input
      type="text"
      value={query}
      onChange={e => setQuery(e.target.value)}
      placeholder="書籍を検索..."
    />
  );
}
```

▼ 動作:

- 検索バー (テキスト入力欄) が表示される
- キーボードで「Re」と打つと query が "" → "R" → "Re" に更新され、画面の入力欄もそれに追従する
- これは **useState** がブラウザのメモリに値を保持し、変更ごとに React が再描画するため
- サーバーはこの入力プロセスにまったく関与しない

重要: **"use client"** はそのファイルとそこから import されるすべてのモジュールを Client Component の境界として宣言します。つまり、Client Component の子コンポーネントも自動的に Client Component になります。

3.3 いつどちらを使うか（判断フローチャート）



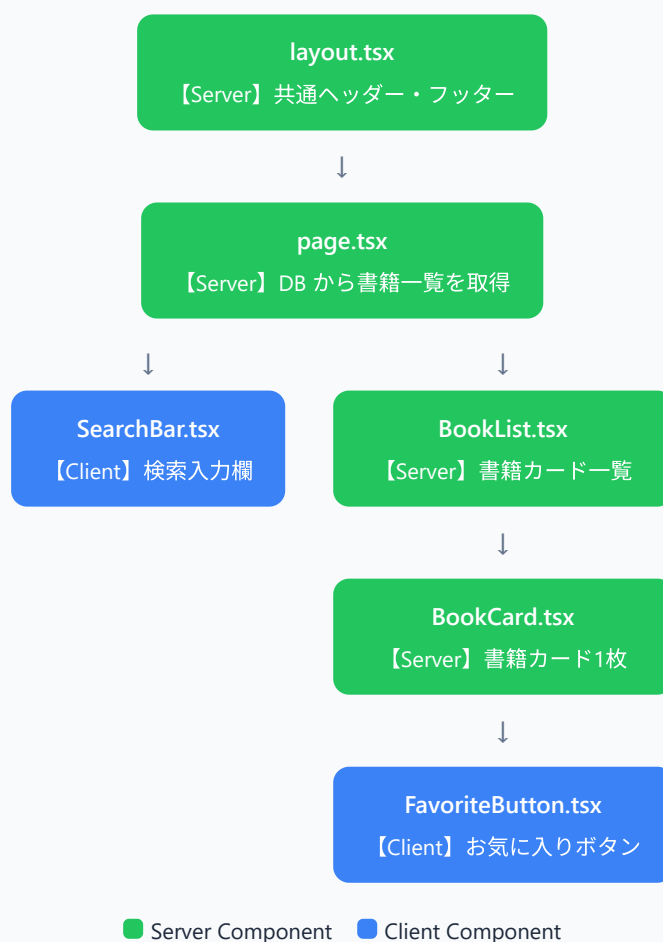
原則: できるだけ Server Component を使い、インタラクティブ性が必要な部分だけを Client Component にします。

用途	Server Component	Client Component
データの取得・表示	推奨	
データベースへの直接アクセス	推奨	不可
機密情報（APIキーなど）へのアクセス	推奨	不可
静的な UI の描画	推奨	
フォーム入力		必須
クリックイベント		必須
useState / useEffect		必須
ブラウザ API（localStorage 等）	不可	必須

3.4 書籍管理アプリでの使い分け例

書籍管理アプリの具体的なページで、どのようにコンポーネントを分割するか見てみましょう。

書籍一覧ページ /books — コンポーネント構成



▼このコードがやること（先に日本語で）：書籍一覧ページ本体を Server Component として作り、サーバー上で書籍データを取得してから画面を組み立てます。ポイントは「Server の骨組み（このページ）の中に、Client の部品（検索バー）と Server の部品（一覧表示）を組み合わせる」という App Router の基本パターンです。取得した **books** を子コンポーネントに props（受け渡しデータ）として渡します。詳細はコード内コメントを参照してください。

```

// app/books/page.tsx (Server Component) // `./books` のページ本
体。"use client" なしなので Server Component
// データベースから直接書籍一覧を取得する

import { SearchBar } from "@components/SearchBar"; // `@/` はプロジェクト
ルートを示すエイリアス // ../../ のような相

// パス地獄を避けるための設定 (tsconfig.json で定義)
import { BookList } from "@components/BookList"; // 書籍リストを描画する
別の自作コンポーネントを取り込む

export default async function BooksPage() { // `async` 関数なので
  本体で `await` が使える // この関数自体が
  Server Component
  // Server Component なので、サーバー上で直接データ取得ができる
  // この処理はブラウザには送信されない
  const response = await fetch("https://api.example.com/books", { // `fetch` は Web標準
    のHTTP通信API // Next.js は fetch
    を拡張してキャッシュ機能を追加している
    cache: "no-store", // 常に最新データを取得 (SSR) // `"no-store"` でキャ
    ッシュを使わず毎回サーバーからデータ取得 // 他に "force-
    cache" (SSG的)、`next: { revalidate: N }` (ISR的) がある
  });
  const books = await response.json(); //
  `response.json()` で本文JSONをJSオブジェクトに変換

  return (
    <div>
      <h1>書籍一覧</h1> // ページタイトル
      { /* Client Component: ユーザーの入力を受け付ける */ } // JSX 内のコメント記
法
      <SearchBar /> // 検索バー
      (Client)。ここから下は子コンポーネント
      { /* Server Component: 取得したデータを表示する */ }
      <BookList books={books} /> // 取得した books 配
列を props として渡す // `books=
{books}` は「booksというpropに変数booksの値を渡す」の意
    </div>
  );
}

```

▼ このコードがやること（先に日本語で）：入力した文字でページを検索結果に切り替える検索バーを作ります。Client Component なので入力状態を `useState` で覚えつつ、`useRouter`（ページ移動用の道具）で検索キーワード付きの URL に移動します。日本語や空白は URL にそのまま使えないため `encodeURIComponent` で変換する点も押さえてください。詳細はコード内コメントを参照してください。

```

// components/SearchBar.tsx (Client Component)
部品
"use client";
ファイルは Client Component

useState やイベントが使える

import { useState } from "react";
ツク
import { useRouter } from "next/navigation";
ーターフック

`next/router` ではない! (あちらは旧 Pages Router 用)

export function SearchBar() {
  `{ SearchBar }` で import される
  const [query, setQuery] = useState("");
  在の検索文字列」を管理

  [現在値, 更新関数] = useState(初期値)
  const router = useRouter();
  ジェクトを取得 (push/replace/back/refresh等が使える)

  const handleSearch = (e: React.FormEvent) => {
    ンドラ。型は React.FormEvent
    e.preventDefault();
    ーム送信 (ページ全体リロード) を止める
    // 検索クエリ付きで遷移
    router.push(`/books?q=${encodeURIComponent(query)}`);
    別URLに移動 (履歴に追加)

    `encodeURIComponent` はURLに使えない文字 (空白や日本語) を %エンコード
  };

  return (
    <form onSubmit={handleSearch}>
    上のハンドラに紐付け
    <input
      type="text"
      value={query}
      ト: 表示値を React state と同期
      onChange={e => setQuery(e.target.value)}
      を更新 (再描画される)
      placeholder="タイトルで検索..."
      テキスト
    />
    <button type="submit">検索</button>
    クリックで form の onSubmit が走る
  </form>

```

```

// 再利用される検索バー
// この行がある=この
// ブラウザ側で動き、
// React の状態管理フ
// App Router 用のル
// ※
// 名前付き export。
// `useState` で「現
// 配列の分割代入:
// ルーター操作オブ
// フォーム送信時のハ
// ブラウザ標準のフォ
// `router.push` で
//
// submit イベントを
// テキスト入力欄
// 制御コンポーネン
// 入力のたびに state
// 空欄時の薄いガイド
// submit ボタン。

```

```

    );
  }

```

▼このコードがやること（先に日本語で）：書籍1冊分の「カード」表示を Server Component で作ります。タイトルや著者など動かない部分はサーバーで描画し、クリックが必要な「お気に入りボタン」だけを Client Component として中に埋め込みます。`type Book = {...}` は「このオブジェクトはこういう形」と決める TypeScript の型定義で、入力ミスを防いでくれます。詳細はコード内コメントを参照してください。

```

// components/BookCard.tsx (Server Component)                // 書籍1冊分のカード。静
// 表示部分は Server で

import { FavoriteButton } from "../FavoriteButton";           // ※同じフォルダから
import するときは "../" で始める                               // ここで Client
                                                                // Component を取り込んでも、子だけが Client になる

type Book = {                                                  // TypeScript の型定
  義 (=この形のオブジェクト)
  id: string;                                                  // 書籍ID (文字列)
  title: string;                                               // タイトル
  author: string;                                              // 著者
};

export function BookCard({ book }: { book: Book }) {           // props を分割代入で
  受け取り、型は `{ book: Book }`
  return (
    <div className="book-card">                                // カード枠 (CSS でス
      タイル付け)
      <h3>{book.title}</h3>                                     // タイトル表示
      <p>{book.author}</p>                                       // 著者表示
      { /* インタラクティブな部分だけ Client Component */ }
      <FavoriteButton bookId={book.id} />                       // お気に入りボタンに
      は id だけ渡す (最小限のpropsで境界を作る)
    </div>
  );
}

```

▼このコードがやること（先に日本語で）：クリックで「お気に入り」の ON/OFF を切り替えるボタンを作ります。Client Component なので、状態を `useState` で覚え、クリックされたらまず画面表示をすぐ切り替え（楽観的更新）、その後サーバーに `fetch` で保存依頼を送ります。ボタンの文字も状態に応じて「☆」と「★」で切り替わります。詳細はコード内コメントを参照してください。

```

// components/FavoriteButton.tsx (Client Component) // クリック可能なお気に入りボタン
"use client"; // この1行で Client Component 化

import { useState } from "react"; // 状態管理用のフック

export function FavoriteButton({ bookId }: { bookId: string }) { // props は `{ bookId: string }`
  const [isFavorite, setIsFavorite] = useState(false); // 「お気に入りか？」の真偽値state。初期値は false

  const handleClick = async () => { // クリック時のハンド
    // async で await が使える
    setIsFavorite(!isFavorite); // まず画面上の状態をトグル (楽観的更新)

    // `!isFavorite`
    // は「現在の値の反対」
    // API を呼び出してお気に入り状態を保存
    await fetch(`/api/books/${bookId}/favorite`, { // テンプレートリテラル (バッククォート ``) で URL を組み立てる
      method: "POST", // HTTP メソッド: POST (新しい状態を送る)
      body: JSON.stringify({ favorite: !isFavorite }), // JSオブジェクトをJSON文字列に変換してリクエストボディに乗せる
    });
  };

  return (
    <button onClick={handleClick}> // ボタンクリックで handleClick を発動
      {isFavorite ? "★ お気に入り済み" : "☆ お気に入り"} // 三項演算子: state によって表示を切り替える
    </button>
  );
}

```

画面にはこう表示される: ページ上部に検索バー、その下に書籍カードが並んで表示されます。各カードには書籍タイトル、著者名、お気に入りボタンがあります。検索バーに文字を入力するとリアルタイムに反映され、お気に入りボタンをクリックすると「☆ お気に入り」が「★ お気に入り済み」に切り替わります。

4. レイアウトとテンプレート

4.1 ルートレイアウト

`app/layout.tsx` は ルートレイアウト と呼ばれ、アプリケーション全体に適用されます。 `<html>` タグと `<body>` タグを含む 唯一の レイアウトです。

▼ このコードがやること（先に日本語で）：アプリ全体の一番外側の枠（ヘッダー・本文・フッター）と、ブラウザタブのタイトルなどの基本情報（ `metadata` ）をまとめて定義します。 `metadata` を export すると Next.js が `<head>` を自動で組み立ててくれるので、自分で `<title>` を書く必要がありません。Google Fonts の最適化読み込みも含む、実用的なルートレイアウトの完成形です。詳細はコード内コメントを参照してください。

```
// app/layout.tsx
下のものは「ルートレイアウト」と呼ばれる

になる（必須）
```

```
import type { Metadata } from "next";
む」記法

ドルが軽くなる
import { Inter } from "next/font/google";
を Next.js 経由で読み込む
```

```
にフォントをセルフホストして高速化してくれる
import "./globals.css";
ート
```

読み込む書き方

```
// Google Fonts の Inter フォントを最適化して読み込む
const inter = Inter({ subsets: ["latin"] });

テン文字だけ含む」指定でサイズ削減
```

```
export const metadata: Metadata = {
すると Next.js が <head> を自動生成

く必要がない
  title: {
    template: "%s | 書籍管理アプリ",
書籍管理アプリ」を付ける
```

```
換わる
    default: "書籍管理アプリ",
ったときのデフォルト
  },
  description: "あなたの読書記録を管理するアプリケーション",
して出力される。SEOで重要
};
```

```
export default function RootLayout({
は慣習で `RootLayout`
  children,
れる「各ページのJSX」
}: {
  children: React.ReactNode;
字列/数値/null など描画できる何でも」を表す型
}) {
  return (
    <html lang="ja">
```

```
// 予約名 `layout.tsx`。`app/` 直
// すべてのページを包む一番外側の枠
```

```
// `import type` は「型だけ取り込
// 実行時のJSには残らないのでバン
// Google Fonts の Inter フォント
// `next/font/google` はビルド時
// 全ページに適用する CSS をインポ
// 値として使わず副作用のためだけに
```

```
// フォントのインスタンスを作る
// `subsets: ["latin"]` は「ラ
// `metadata` という名前で export
// 自分で <title> や <meta> を書
// 子ページのタイトルに自動で「 |
// `%s` の部分が子の title に置き
// 子ページが title を指定しなか
```

```
// <meta name="description"> と
// ルートレイアウトの本体関数。名前
// children: このレイアウトに包ま
// `React.ReactNode` は「JSX/文
// `` はルートレイアウトでだ
```

け書く

ことをブラウザや支援技術に伝える

```
<body className={inter.className}>
```

用 CSS クラスを適用

```
{/* ここにヘッダーを配置 */}
```

```
<header className="site-header">
```

ページ上部のヘッダー領域

```
<nav>
```

すセマンティック要素

```
<a href="/">書籍管理アプリ</a>
```

(後で `<Link>` に置き換えるのがベター)

```
</nav>
```

```
</header>
```

```
{/* children に各ページの内容が入る */}
```

```
<main className="site-main">{children}</main>
```

領域

`'app/books/page.tsx'` 等の中身が差し込まれる

```
{/* ここにフッターを配置 */}
```

```
<footer className="site-footer">
```

一領域

```
<p>&copy; 2024 書籍管理アプリ</p>
```

ィティ (特殊文字)

```
</footer>
```

```
</body>
```

```
</html>
```

```
);
```

```
}
```

```
// lang="ja" は日本語ページである
```

```
// '<body>' もルートレイアウト専用
```

```
// 'inter.className' でフォント
```

```
// セマンティック要素 '<header>':
```

```
// '<nav>': ナビゲーション領域を示
```

```
// 普通のアンカー。ロゴ的リンク
```

```
// '<main>': ページの主要コンテンツ
```

```
// ここに 'app/page.tsx' や
```

```
// '<footer>': ページ下部のフッタ
```

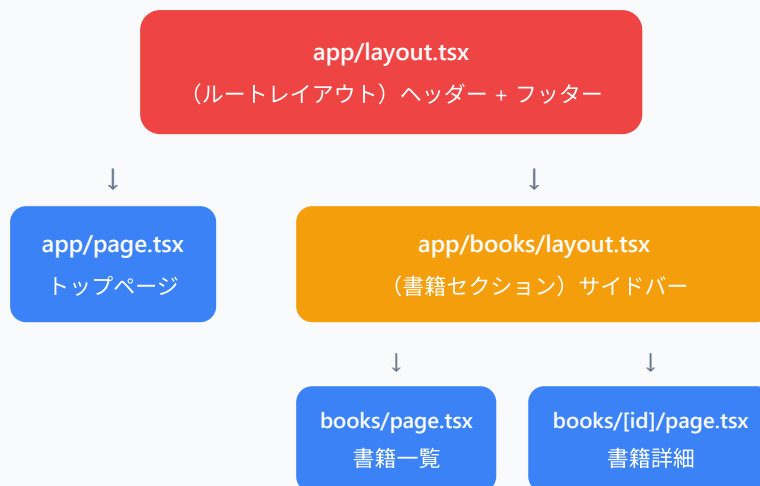
```
// '&copy;' は © の HTML エンテ
```

ポイント: - ルートレイアウトは **必須** です。削除するとエラーになります。 - `<html>` と `<body>` タグはルートレイアウトにのみ記述します。 - `metadata` オブジェクトで SEO 用のタイトル・説明を設定できます。 - `template: "%s | 書籍管理アプリ"` により、子ページのタイトルが「ページ名 | 書籍管理アプリ」の形式になります。

4.2 ネストされたレイアウト

各ルートセグメント (フォルダ) にも `layout.tsx` を置くことができ、そのセグメント以下のページに適用されます。

レイアウトのネスト構造



▼ このコードがやること（先に日本語で）： `/books` 以下のページだけに共通で付くサイドバー付きの枠（ネストされたレイアウト）を作ります。ルートレイアウトの内側にさらに重ねられるのがポイントで、トップページには出ず `/books` 配下でだけサイドバーが表示されます。ルート以外のレイアウトでは `<html>` / `<body>` は書かない点に注意してください。詳細はコード内コメントを参照してください。

```

// app/books/layout.tsx                                // `app/books/` フォルダの中
の `layout.tsx`
// /books 以下のすべてのページに適用されるレイアウト    // `/books`, `/books/new`,
`/books/[id]` 等で共通の枠になる

import type { Metadata } from "next";                  // 型だけ import (Metadata 型
を使うため)

export const metadata: Metadata = {                      // この階層用のメタデータ
  title: "書籍管理",                                    // 親 (RootLayout) の
template と組み合わせり「書籍管理 | 書籍管理アプリ」になる
};

export default function BooksLayout({                    // ネストされたレイアウトの本体
  children,                                              // children: この階層の
  page.tsx またはさらに深い layout.tsx の中身
}: {
  children: React.ReactNode;                            // 描画可能な何でも、を意味する
  型
}) {
  return (
    <div className="books-layout">                      // ※ルート以外の layout.tsx
    は <html>/<body> を書かない
      { /* サイドバー */
        <aside className="sidebar">                    // `<aside>`: 補足コンテンツ用
        のセマンティック要素
          <nav>
            <ul>
              <li><a href="/books">書籍一覧</a></li>      // ナビゲーション領域
              // 箇条書きリスト
              // ※ 後ほど `<Link>` への
              置き換えを学ぶ
              <li><a href="/books/new">書籍を追加</a></li>
              <li><a href="/books/favorites">お気に入り</a></li>
            </ul>
          </nav>
        </aside>

        { /* メインコンテンツ */
          <section className="content">                  // `<section>`: 意味的にひと
          まとまりの領域
            {children}                                    // ここに /books/page.tsx や
            </section>
          </div>
        );
      }

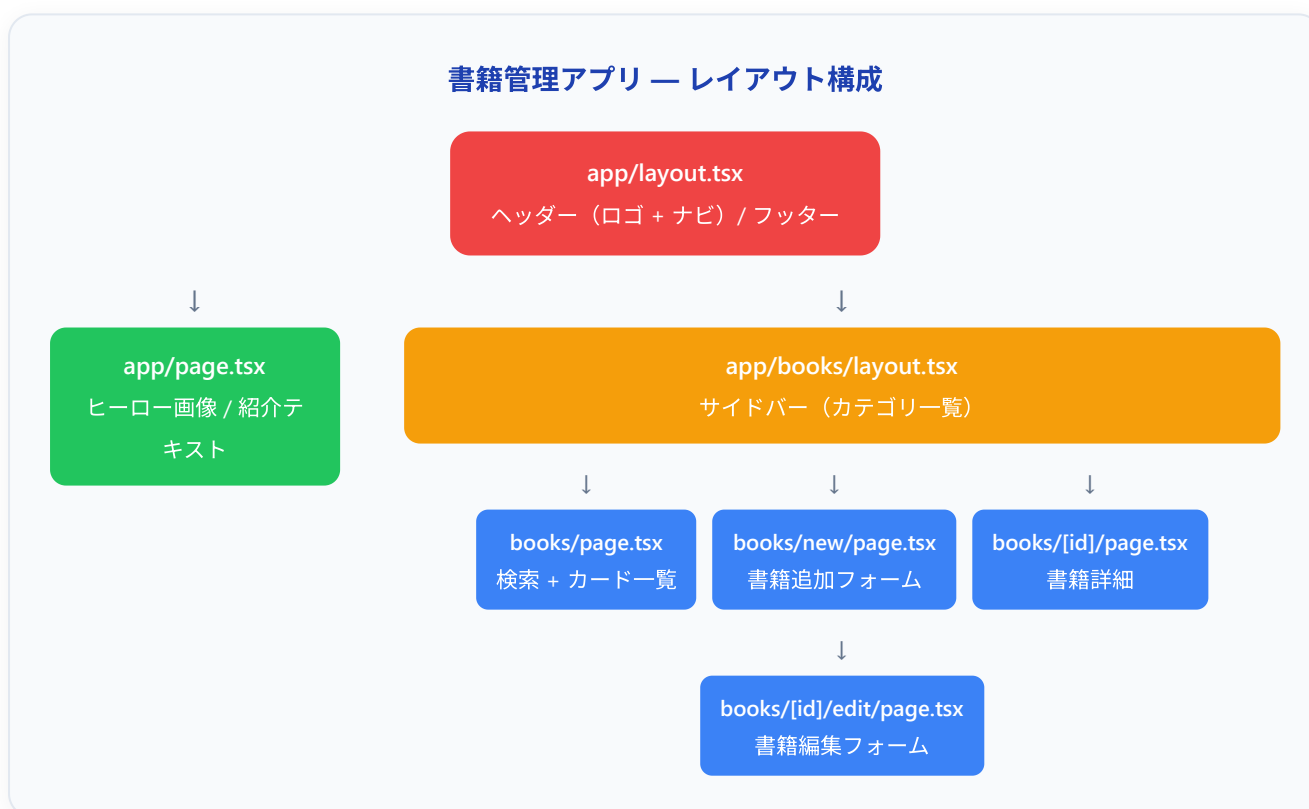
```

画面にはこう表示される:



`/books` や `/books/123` にアクセスすると、ルートレイアウトのヘッダー・フッターに加えて、書籍セクション専用のサイドバーが表示されます。トップページ (`/`) にはサイドバーは表示されません。

4.3 書籍管理アプリのレイアウト構成



5. ナビゲーション

5.1 Link コンポーネント

Next.js では、ページ間の遷移に HTML の `<a>` タグではなく、`next/link` モジュールが提供する `<Link>` コンポーネントを使います。`<Link>` は内部的に `<a>` を描画しますが、クリック時にページ全体をリロードせず、必要な部分だけを差し替える **クライアントサイド遷移**（client-side navigation：ブラウザでURLだけ書き換えて描画を切り替える方式）を行います。

▼ このコードがやること（先に日本語で）：ページ間を移動するためのナビゲーション（メニュー）を、`next/link` の `<Link>` を使って作ります。`<a>` タグの代わりに `<Link>` を使うと、クリック時にページ全体を読み込み直さず必要な部分だけ差し替える「高速なページ移動」になります。`href` に行き先 URL を指定する書き方は `<a>` とよく似ています。詳細はコード内コメントを参照してください。

```
// components/Navigation.tsx                                // ナビゲーション専用コンポーネント
import Link from "next/link";                                // Next.js の Link コンポーネントを取
// リ込む (default export)

export function Navigation() {                                // 名前付き export (中括弧で import
// される)
  return (
    <nav>                                                       // セマンティック要素：ナビゲーション領
// 域
    <ul>                                                         // リスト
      <li>                                                         // 1項目
        <Link href="/">ホーム</Link>                            // `href` で行き先URLを指定。<a> と
// 似た書き方
      </li>
      <li>
        <Link href="/books">書籍一覧</Link>                    // `/books` への高速遷移リンク
      </li>
      <li>
        <Link href="/books/new">書籍を追加</Link>              // `/books/new` への遷移
      </li>
    </ul>
  </nav>
);
}
```

`<Link>` と `<a>` の違い:

特性	<code><a></code> タグ	<code><Link></code> コンポーネント
ページ遷移	ページ全体をリロード	クライアント側で遷移（高速）
プリフェッチ	なし	ビューポート内のリンクを自動プリフェッチ
状態の保持	リセットされる	レイアウトの状態が保持される
JavaScript	不要	必要

▼ このコードがやること（先に日本語で）：書籍ごとに異なる詳細ページ（例: `/books/42`）へのリンクを、データに応じて動的に組み立てます。`href={ /books/${book.id} }` のようにバッククォート（テンプレートリテラル）で URL に変数を埋め込むのがポイントです。1冊分の `book` を `props` で受け取り、その `id` からリンク先を作ります。詳細はコード内コメントを参照してください。

```
// 動的ルートへのリンク
// リンク例
import Link from "next/link";

type Book = {
  id: string;
  title: string;
};

export function BookLink({ book }: { book: Book }) {
  return (
    <Link href={`/${books}/${book.id}`}>
      {book.title} の詳細を見る
    </Link>
  );
}
```

```
// 動的セグメント `[id]` を含むURLへの
// Link コンポーネントを取り込む
// 書籍の型を簡易定義
// 書籍ID
// タイトル
// props として 1冊の book を受け取る
// テンプレートリテラルで URL を組み立てる
// 例: book.id が "42" なら
// 表示テキストは book.title を埋め込み
```

画面にはこう表示される: 「○○の詳細を見る」というリンクテキストが表示されます。クリックすると、ページ全体がリロードされることなく、スムーズに書籍詳細ページに遷移します。ヘッダーやサイドバーのレイアウトはそのまま残り、メインコンテンツ部分だけが切り替わります。

プリフェッチについて: `<Link>` コンポーネントはデフォルトで、ユーザーの画面（ビューポート）に表示されているリンク先のページを裏で先読みします。これにより、リンクをクリックした瞬間にページが表示されるような高速な体験が実現します。

5.2 useRouter フック

プログラムからページ遷移を行いたい場合（ボタンクリック後やフォーム送信後など）は、`useRouter` フックを使います。Client Component でのみ使用可能です。

▼ このコードがやること（先に日本語で）： `<Link>` のクリックではなく「処理が終わった後にプログラムから自動でページを移動する」方法を学びます。フォーム送信後に `useRouter` の `router.push("/books")` を呼んで一覧ページへ飛ばす例です。`useRouter` は Client Component でしか使えず、取り込み元は `next/navigation`（旧 `next/router` ではない）である点が要注意です。詳細はコード内コメントを参照してください。

```
// components/BookForm.tsx                                     // 書籍フォーム部品
"use client";                                                  // useRouter は Client
Component でしか使えないため必須

import { useRouter } from "next/navigation"; // ⚠️ next/router ではない! // App Router
用は `next/navigation`、旧 Pages Router 用は `next/router`

export function BookForm() {
  const router = useRouter();                                // ルーターインスタンスを取得

  const handleSubmit = async (e: React.FormEvent) => { // フォーム送信時のハンドラ
    e.preventDefault();                                    // デフォルトの送信動作（ページリロ
    ード）を防ぐ

    // 書籍を保存する処理（省略）                          // 例: await fetch("/api/books",
    { method: "POST", ... });

    // 保存後、書籍一覧ページに遷移
    router.push("/books");                                // プログラム的に画面遷移。履歴に
    「/books」が追加される
  };

  return (
    <form onSubmit={handleSubmit}>                          // submit イベントを上記関数に紐付
    け
      { /* フォームの内容（省略） */ }                      // 実際には input 要素などを並べる
      <button type="submit">保存</button>                  // submit ボタン
    </form>
  );
}
```

useRouter の主要メソッド:

▼このコードがやること（先に日本語で）：**useRouter** が持つ4つの代表的な機能を、ボタンで呼び分けて比較します。**push**（履歴を残して移動）、**replace**（履歴を上書きして移動＝戻れない）、**back**（1つ前に戻る）、**refresh**（今のページのデータだけ取り直す）の違いを押さえるのが目的です。それぞれをいつ使うかはこの後の表にもまとまっています。詳細はコード内コメントを参照してください。

```
"use client"; // Client Component 化

import { useRouter } from "next/navigation"; // App Router 用のフック

export function NavigationExample() {
  const router = useRouter(); // ルーターを取得

  return (
    <div>
      {/* 指定したパスに遷移（履歴に追加） */}
      <button onClick={() => router.push("/books")}> // `push`: 履歴を1つ増やしなが
遷移（ブラウザの戻るで前ページに戻れる）
        書籍一覧へ
      </button>

      {/* 指定したパスに遷移（履歴を置換） */}
      <button onClick={() => router.replace("/books")}> // `replace`: 現在の履歴エントリ
を上書きして遷移（戻れない）
        書籍一覧へ（履歴を置換） // ログイン後にログインページを履歴
に残したくないときに使う
      </button>

      {/* ブラウザの「戻る」と同じ */}
      <button onClick={() => router.back()}> // `back`: 履歴を1つ戻る
        戻る
      </button>

      {/* ページのデータを再取得（再レンダリング） */}
      <button onClick={() => router.refresh()}> // `refresh`: 現在のルートの
Server Component を再実行・再描画
        更新 // ※ ページ全体リロードではない
        (state は保持される)
      </button>
    </div>
  );
}
```

メソッド	説明	使用例
<code>router.push(url)</code>	指定した URL に遷移	フォーム送信後のリダイレクト
<code>router.replace(url)</code>	現在の履歴エントリを置換して遷移	ログイン後のリダイレクト
<code>router.back()</code>	ブラウザの戻るボタンと同じ	詳細ページから一覧に戻る
<code>router.refresh()</code>	現在のページを再取得	データ更新後の再読み込み
<code>router.prefetch(url)</code>	指定した URL を先読み	近く遷移しそうなページの準備

5.2.1 usePathname / useSearchParams (補足)

`next/navigation` には `useRouter` 以外にも、現在の URL に関する情報を取り出すフックがあります。どちらも **Client Component 専用** です。

▼ このコードがやること（先に日本語で）：「今どの URL にいるか」「URL の `?sort=price` のような追加情報（クエリパラメータ）は何か」をプログラムから読み取ります。`usePathname` で現在のパス、`useSearchParams` でクエリの値（`.get("sort")` で取得）を取り出す例です。`??`（ヌル合体演算子）は「値が無いときの代わりの値」を指定する書き方です。詳細はコード内コメントを参照してください。

```
"use client"; // Client Component
必須

import { usePathname, useSearchParams } from "next/navigation"; // 2つのフックを取り込む

export function CurrentUrlInfo() {
  const pathname = usePathname(); // 例: "/books/123" の
  ような現在のパス（クエリは含まない）
  const searchParams = useSearchParams(); // クエリパラメータを扱
  う Readonly な URLSearchParams 風オブジェクト

  const sort = searchParams.get("sort"); // `?sort=price` なら
  "price" を返す。なければ null
  const page = searchParams.get("page") ?? "1"; // `??` は左辺が
  null/undefined のとき右辺を使う「ヌル合体演算子」

  return (
    <p>
      現在のパス: {pathname}（並び順: {sort ?? "デフォルト"}、ページ: {page}）
    </p>
  );
}
```

5.3 リダイレクト

リダイレクト（redirect：別のURLに自動転送すること）は、Server Component の中で `redirect()` を呼ぶか、後述する `middleware.ts` で行います。

Server Component でのリダイレクト

▼ このコードがやること（先に日本語で）：書籍詳細ページで「条件によって別ページへ自動転送する」処理を学びます。データが見つからなければ `notFound()` で404ページを、非公開の書籍なら `redirect("/books")` で一覧へ飛ばす例です。どちらも呼ぶとそこで処理が止まる（戻ってこない）ため、`return` の前に置くだけで分岐できます。詳細はコード内コメントを参照してください。

```

// app/books/[id]/page.tsx                                // 動的セグメントを含む書
籍詳細ページ
import { redirect, notFound } from "next/navigation";      // 2つのヘルパー関数を取
り込む
                                                            // redirect: 任意の
URL へ転送
                                                            // notFound: 404扱い
にして not-found.tsx を表示

export default async function BookDetailPage({              // async Server
  Component
  params,
}: {
  params: Promise<{ id: string }>;                          // Next.js 15+ では
  params は Promise でラップ
}) {
  const { id } = await params;                               // await で id を取り出
す                                                         // 仮想のデータ取得関数
  const book = await fetchBook(id);

  // 書籍が見つからない場合は 404 ページを表示
  if (!book) {                                              // データが
null/undefined なら
    notFound();                                           // この呼び出しで以降の
                                                         // 関数の戻り値は
never (戻ってこない関数の型)
  }

  // 非公開の書籍は一覧ページにリダイレクト
  if (book.isPrivate) {                                    // 条件次第で
    redirect("/books");                                   // `/books` に転送 (こ
れも never を返す)
  }

  return (                                                 // 上の if を通り抜けた
    場合だけここに到達
    <div>
      <h1>{book.title}</h1>                               // タイトル表示
    </div>
  );
}

async function fetchBook(id: string) {                    // データ取得を模した関
数
  // データベースから書籍を取得する処理 (仮
  呼び出し                                              // 本来は DB クエリや API

```

```
return { title: "サンプル書籍", isPrivate: false }; // ダミーデータ
}
```

middleware.ts でのリダイレクト

`middleware.ts`（ミドルウェア：リクエストとレスポンスの間に挟まる処理。Next.js では Edge Runtime（軽量な実行環境）で動く）をプロジェクトルートに置くことで、ページが描画される**前の段階**でリダイレクトを行えます。認証チェック、地域別の振り分け、A/Bテストなどに使えます。

▼ このコードがやること（先に日本語で）：ページが表示される「前の段階」で割り込み、未ログインのユーザーを `/login` に転送する関所（ミドルウェア）を作ります。リクエストの Cookie からログイン状態を判定し、条件に合えば転送、合わなければそのまま通します。最後の `config.matcher` で「どの URL にこの関所を効かせるか」を指定する点もポイントです。詳細はコード内コメントを参照してください。

```

// middleware.ts (プロジェクトルート)                                // ファイル名は固定。app/
の外、プロジェクト直下に置く

import { NextResponse } from "next/server";                        // Edge ランタイム用の
レスポンスヘルパー

import type { NextRequest } from "next/server";                    // 型だけ取り込み（リク
エスト型）

export function middleware(request: NextRequest) {                  // `middleware` という
名前 export すると Next.js が自動呼び出し
  // 例: 未ログインユーザーを /books ページからリダイレクト
  const isLoggedIn = request.cookies.get("session");                // Cookie から
  `session` の値を取得（あればログイン中とみなす）

  if (!isLoggedIn && request.nextUrl.pathname.startsWith("/books")) { // 未ログイン かつ
    /books 配下にアクセスしている
    return NextResponse.redirect(new URL("/login", request.url)); // /login に転送するレ
    スポンスを返す
                                                                    // `new
    URL("/login", request.url)` はベースURLを保ったまま絶対URLを作る
  }

  return NextResponse.next();                                        // 条件に合わない場合は
  普通に次の処理（=ページ描画）へ進める
}

// ミドルウェアを適用するパスを指定
export const config = {                                            // `config` という名前
  // 設定オブジェクトを export
  matcher: ["/books/:path*"],                                       // この matcher にマッ
  // ちしたURLでだけ middleware が動く
                                                                    // `:path*` は「任意
  // の階層を含む」の意味
};

```

6. データフェッチ

6.1 Server Component でのデータ取得

Server Component ではコンポーネント関数を `async` にして、直接 `await` でデータを取得できます。これが Next.js App Router の最も強力な機能の1つです。

▼ このコードがやること（先に日本語で）：Server Component の最大の魅力である「コンポーネント関数を `async` にして、その場で `await` でデータを取得する」書き方を学びます。取得した書籍配列を `map`（1件ずつ変換）でリスト表示する、データ取得の王道パターンです。データが入った状態の HTML がサーバーから届くので表示が速く SEO にも強い、という普通の React との違いも意識してください。詳細はコード内コメントを参照してください。


```
// =====
// ファイルパス: app/books/page.tsx
// 役割      : 書籍一覧ページ。サーバーで全書籍を取ってきてHTMLにする。
// 種類      : Server Component ("use client" を書いていないので自動でこちら)
// -----

// Server Component の最大の魅力は「コンポーネント関数自体を async にできて、
// 直接 await でデータを取ってこられる」こと。
// そのデータはサーバーで埋め込まれた状態でブラウザに届くので、
// 初回表示が速く、SEOにも有利。
// =====

// (1) 書籍データ1件分の「形」を定義
//      Book 型を1度ここで決めておくと、map のコールバック内などで
//      book.〇〇 と書いたとき VS Code が補完してくれる。
type Book = {
  id: string;           // 一意なID
  title: string;        // 書籍タイトル
  author: string;       // 著者名
  publishedYear: number; // 出版年
};

// (2) APIから書籍配列を取ってくる関数
//      async を付けると、関数の中で await が使えるようになる。
//      戻り値は「Book型の配列が将来届く」を意味する Promise<Book[]>。
async function getBooks(): Promise<Book[]> {

  // fetch は Web標準のHTTP通信API。
  // 第2引数の cache オプションで Next.js のキャッシュ動作を指定する。
  const response = await fetch("https://api.example.com/books", {
    cache: "no-store",    // ← 毎リクエストで最新データを取得する (=SSR動作)
    // cache: "force-cache",    // ← ビルド時に1度だけ取得 (=SSG動作)
    // next: { revalidate: 60 }, // ← 60秒ごとに再取得 (=ISR動作)
  });

  // (2-1) 通信は成功したけどステータスが 4xx/5xx の場合は失敗扱いにする。
  // response.ok は status が 200~299 のとき true、それ以外で false。
  if (!response.ok) {
    throw new Error("書籍データの取得に失敗しました");
  }

  // (2-2) JSON 文字列を JS オブジェクトに変換して返す
  return response.json();
}

// (3) ページコンポーネント本体
//      関数自体に async を付けられるのが Server Component の特権。
```

```
// Client Component ("use client"あり) では関数本体を async にできない。
export default async function BooksPage() {

  // この行はサーバーで実行される。
  // 取得処理が終わるまでブラウザにはHTMLが送られない（その間 loading.tsx が表示される）。
  const books = await getBooks();

  // (4) 取れたデータを使ってJSXを組み立てる
  // books.length は配列の要素数（書籍が3件なら3）。
  // books.map((book) => ...) で1件ずつJSXに変換する。
  // key={book.id} は React がリストの差分検出に使う必須の属性。
  return (
    <div>
      <h1>書籍一覧（{books.length}冊）</h1>
      <ul>
        {books.map((book) => (
          <li key={book.id}>
            <h3>{book.title}</h3>
            <p>著者: {book.author}（{book.publishedYear}年）</p>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

▼ APIが3件のデータを返した場合のブラウザ表示:

書籍一覧（3冊）

- ・ リーダブルコード
著者: Dustin Boswell（2012年）
- ・ プロを目指す人のためのTypeScript入門
著者: 鈴木 僚太（2022年）
- ・ 達人プログラマー
著者: David Thomas（2016年）

▼ 通信失敗時:

`throw new Error(...)` が走ると、Next.js は同じセグメント内の `error.tsx` を自動で表示します。 `error.tsx` を作っていない場合は親のエラーバウンダリにバブリングします。

▼ ブラウザに届くHTMLの様子（要点だけ抜粋）:

```

<div>
  <h1>書籍一覧（3冊）</h1>
  <ul>
    <li><h3>リーダブルコード</h3><p>著者: Dustin Boswell（2012年）</p></li>
    <li><h3>プロを目指す人のためのTypeScript入門</h3><p>著者: 鈴木 僚太（2022年）</p></li>
    <li><h3>達人プログラマー</h3><p>著者: David Thomas（2016年）</p></li>
  </ul>
</div>

```

▼ 普通のReactとの最大の違い: クライアントサイドの `useState/useEffect` でデータを取りに行くのと違って、最初からデータが入ったHTMLが返ってくるので、表示がカクカクしない・SEOに強い・JSバンドルも小さく済む。

fetch のキャッシュ戦略

Next.js の `fetch` には、標準仕様にはない独自のオプションが追加されています。これらを使い分けることで SSR / SSG / ISR（インクリメンタル静的再生成）を切り替えられます。

```

// SSR - リクエストのたびに最新データを取得                      // 毎回サーバー
// 最新データを取りに行く動作
fetch(url, { cache: "no-store" });                                // `cache:
`no-store` でキャッシュを使わない

// SSG - ビルド時にデータを取得し、キャッシュ（デフォルト）      // ビルド時に1
// 回だけ取得し、以降は同じ結果を返す
fetch(url, { cache: "force-cache" });                             // `cache:
`force-cache` で強制キャッシュ

// ISR - 60秒ごとにキャッシュを再検証                              // 一定間隔で
// 再取得する中間方式
fetch(url, { next: { revalidate: 60 } });                          //
`next.revalidate` は Next.js が追加した独自オプション

// 60 は秒
数。指定間隔ごとに次のアクセス時に再取得

```

戦略	説明	使用例
<code>cache: "no-store"</code>	毎回取得（SSR）	ユーザー固有のデータ
<code>cache: "force-cache"</code>	ビルド時に取得（SSG）	変更の少ないデータ
<code>next: { revalidate: N }</code>	N秒ごとに再検証（ISR）	ニュース記事など

6.2 Client Component でのデータ取得

Client Component では、従来の React と同じように `useEffect` と `useState` を使うか、サードパーティのデータ取得ライブラリ（SWR や TanStack Query）を使います。

▼ このコードがやること（先に日本語で）：ブラウザ側でリアルタイムに検索する部品を、従来の React と同じ `useState` + `useEffect` で作ります。入力のたびに毎回通信すると重いので、「入力が止まって300ミリ秒後にだけ検索する」デバウンスという工夫を入れています。読み込み中・エラー・結果という3つの状態をそれぞれ管理する、実戦的な例です。詳細はコード内コメントを参照してください。

```

// components/BookSearch.tsx
ム検索する部品
"use client";
Component。useState/useEffect を使うため必須

import { useState, useEffect } from "react";
のフック

type Book = {
  id: string;
  title: string;
  author: string;
};

export function BookSearch() {
  const [query, setQuery] = useState("");
  (初期値は空文字)
  const [books, setBooks] = useState<Book[]>([]);
  エネリック `<Book[]>` で要素の型を指定
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  (なければ null)

  useEffect(() => {
    走る副作用
    // 検索クエリが空なら何もしない
    if (!query.trim()) {
      空白を取り除いて空かチェック
      setBooks([]);
      return;
    }

    // デバウンス: 入力が止まって300ms後に検索を実行
    const timer = setTimeout(async () => {
      マーを仕掛ける

      はクリーンアップで毎回キャンセルされる (=デバウンス)
      setIsLoading(true);
      リ替え
      setError(null);
      ト

      try {
        処理を try で囲む
        const response = await fetch(
          エスト
          `/api/books/search?q=${encodeURIComponent(query)}`
          に検索文字列を入れる
        );

```

```

// ブラウザでリアルタイ
// Client
// 状態管理と副作用管理
// 検索結果1件の型
// 書籍ID
// タイトル
// 著者
// 検索文字列の state
// 検索結果の配列。ジ
// 読み込み中フラグ
// エラーメッセージ
// query が変わるたび
// `trim()` で前後の
// 空の結果に
// ここで打ち切り
// 300ms後に走るタイ
// 入力が続いている間
// 「読み込み中」に切
// エラー表示はリセッ
// 例外の可能性のある
// 内部APIに GET リク
// クエリパラメータ q

```

```

        if (!response.ok) { // HTTP ステータスが
200番台でなければ
            throw new Error("検索に失敗しました"); // 例外を投げて
        catch に飛ばす
        }
        const data = await response.json(); // レスポンス本文
        (JSON) を JS オブジェクトに変換
        setBooks(data); // state に保存 → 再
        描画
    } catch (err) { // 通信失敗や上の
        throw を捕まえる
        setError(err instanceof Error ? err.message : "エラーが発生しました");
        // err が Error 型
        ならその message を、それ以外なら汎用文を表示
        } finally { // 成功・失敗どちらで
        も最後に走る
        setIsLoading(false); // 読み込み中フラグを
        解除
        }
    }, 300); // 300ms = デバウン
    ス時間

    // クリーンアップ: 次の入力 cameたらタイマーをキャンセル
    return () => clearTimeout(timer); // useEffect が
    return する関数は「次のeffect実行前」または「unmount時」に呼ばれる
    }, [query]); // 依存配列。query
    が変わったときだけ effect を再実行

    return (
        <div>
            <input
                type="text" // 1行テキスト入力欄
                value={query} // 表示値を state と
                onChange={e => setQuery(e.target.value)} // 入力変更時に state
                placeholder="書籍を検索..." // 空欄ガイド
            />

            {isLoading && <p>検索中...</p>} // 短絡評価:
            isLoading が true のときだけ <p> を描画
            {error && <p className="error">{error}</p>}} // エラー文字列がある
            とときだけ表示

            <ul>
                {books.map((book) => ( // 配列を JSX 要素に
                マッピング
                <li key={book.id}> // `key` は React に
                「どの項目が同じか」を伝える必須属性
                {book.title} - {book.author} // タイトル - 著者 の

```

```
形式で表示
    </li>
  })}
</ul>
</div>
);
}
```

画面にはこう表示される: 検索入力欄が表示されます。ユーザーが「村上」と入力すると、入力が止まって300ミリ秒後に「検索中...」というメッセージが一瞬表示され、その後「村上春樹」の著書一覧がリスト形式で表示されます。

Server Component と Client Component のデータ取得の比較:

特性	Server Component	Client Component
コード量	少ない (async/await のみ)	多い (useState + useEffect)
ローディング状態	loading.tsx が自動管理	自分で管理が必要
エラー処理	error.tsx が自動管理	自分で管理が必要
データの安全性	APIキー等が露出しない	APIキーは使えない
SEO	HTML にデータが含まれる	初回は空
リアルタイム更新	ページ再読み込みが必要	可能

6.3 loading.tsx による読み込み状態

loading.tsx を配置するだけで、Server Component のデータ取得中に自動的にローディング UI が表示されます。

▼ このコードがやること (先に日本語で) : 読み込み中に、実際のコンテンツと同じ形をした灰色の枠 (スケルトン UI) を表示するローディング画面を作ります。データが届いた瞬間に画面がガタつかないようにするための工夫です。
Array.from({ length: 6 }).map(...) で同じカードを6個並べる書き方や、**style** 属性にオブジェクトを渡す JSX 特有の書き方を学べます。詳細はコード内コメントを参照してください。

```

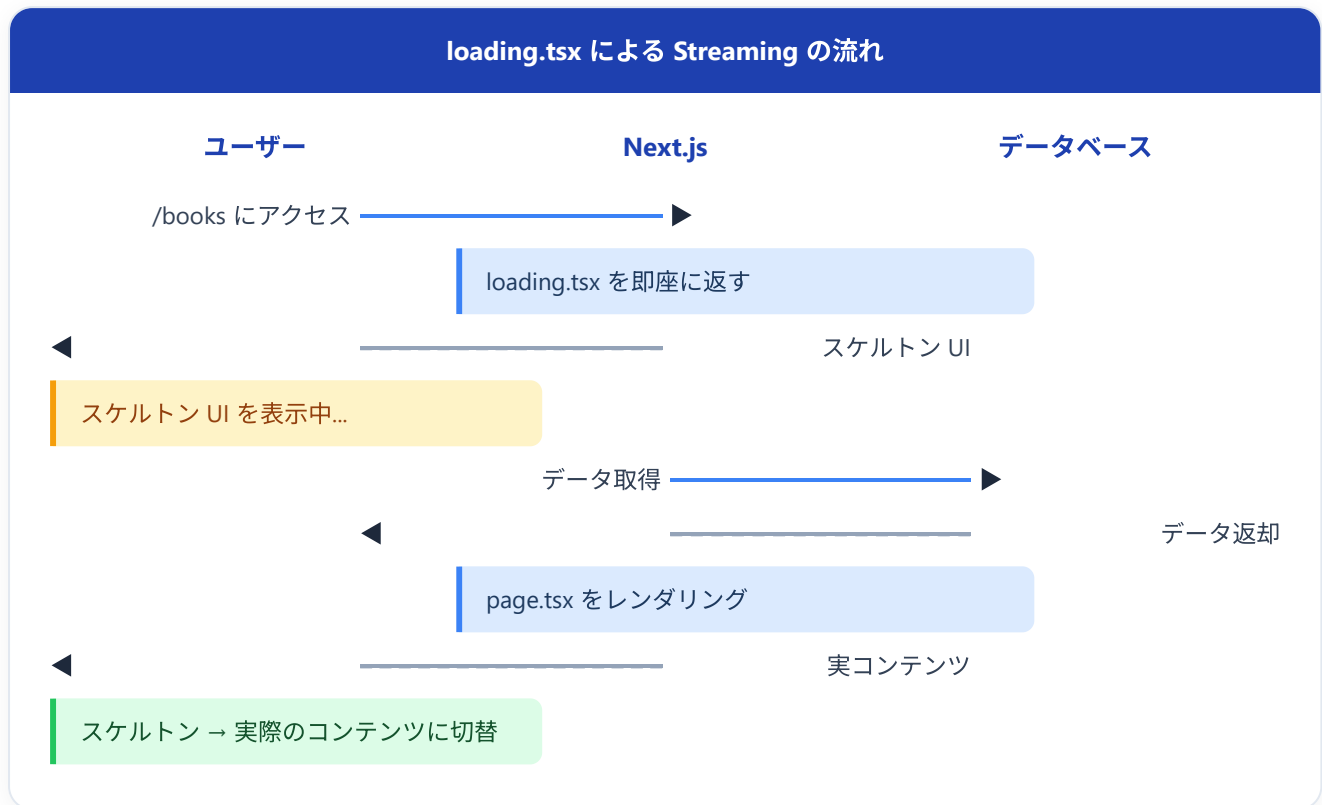
// app/books/loading.tsx // 予約名
`loading.tsx`。/books のロード中UI

export default function BooksLoading() { // default export 関
  数
  return (
    <div className="loading-container">
      {/* スケルトンUI: 実際のコンテンツと同じ形状の灰色の枠 */}
      <h1 className="skeleton" style={{ width: "200px", height: "32px" }} />
      // `style` 属性はオブ
      ジェクトを渡す (CSSのキャメルケース表記)
      // `{{ ... }}` の外
      側はJSXの式、内側はJSオブジェクト

      <div className="book-grid">
        {/* 6個のスケルトンカード */}
        {Array.from({ length: 6 }).map((_, i) => ( // `Array.from({
length: 6 })` で要素6個の配列を作る
          // 第1引数 `_` は使わ
          // 第2引数 `i` は
          0,1,2,...,5 のインデックス
          <div key={i} className="skeleton-card"> // key にインデックス
            を使う (並び順が変わらないリストならOK)
            <div
              className="skeleton"
              style={{ width: "100%", height: "200px" }} // 表紙画像の代わりの
              灰色ブロック
            />
            <div
              className="skeleton"
              style={{ width: "80%", height: "20px", marginTop: "8px" }} // タイトル代わりの細
              長い灰色ブロック
            />
            <div
              className="skeleton"
              style={{ width: "60%", height: "16px", marginTop: "4px" }} // 著者代わりのさらに
              小さなブロック
            />
          </div>
        )}
      </div>
    </div>
  );
}

```


画面にはこう表示される: データの読み込み中、見出しの位置に灰色のバーが1つ、その下に灰色のカード状のブロックが6個並びます。これらは実際のコンテンツと同じ大きさ・位置で表示されるため、読み込みが完了するとスムーズに実際のコンテンツに切り替わり、画面がガタつきません（この手法を **スケルトンUI** と呼びます）。



6.4 Route Handlers（補足）

App Router で「自前のHTTP API」を作りたいときは、`app/api/.../route.ts` というファイルを置きます。これを **Route Handler**（ルートハンドラ：URLとHTTPメソッドに対応する関数を書くファイル）と呼びます。Server Component から直接DBにアクセスできる App Router では出番は減りましたが、外部から `fetch` で叩かれるエンドポイントを公開したい場合に使います。

▼ このコードがやること（先に日本語で）：`app/api/.../route.ts` という予約名のファイルで、外部から `fetch` で呼べる自前の HTTP API を作ります。`GET` や `POST` という名前の関数を `export` すると、それぞれが同名の HTTP メソッドの処理になるのがポイントです。`NextResponse.json(...)` で JSON を返す、API の基本形を学べます。詳細はコード内コメントを参照してください。

```

// app/api/books/route.ts // 予約名 `route.ts`。
`/api/books` の HTTP API になる // page.tsx と同じフォ

ルダには置けない（ルートが衝突するため）

import { NextResponse } from "next/server"; // レスポンスを作るヘル
パー

export async function GET() { // 関数名 `GET` が HTTP
  メソッドに対応（POST/PUT/DELETE等も同様）
  const books = [ // 仮データ
    { id: "1", title: "リーダブルコード" },
    { id: "2", title: "達人プログラマー" },
  ];
  return NextResponse.json(books); // 配列を JSON にしてレ
  スポンスとして返す
}

export async function POST(request: Request) { // POST メソッド用。引数
  は Web 標準の Request
  const body = await request.json(); // リクエストボディの
  JSON を解釈
  // ここで body を使ってDB保存などを行う
  return NextResponse.json({ created: true, body }, { status: 201 });
  // 201 Created を返す
}
例
}

```

6.5 next/image（補足）

画像は `next/image` の `<Image>` コンポーネントを使うと、自動でサイズ最適化・WebP変換・遅延ロードが行われます。

▼ このコードがやること（先に日本語で）：画像を `<img>` ではなく `next/image` の `<Image>` で表示し、サイズ最適化・WebP変換・遅延読み込みを自動で効かせます。`width / height` はレイアウト崩れ防止のために必須、`alt`（代替テキスト）もアクセシビリティのために必須です。最重要画像には `priority` を付ける、という使い方を学べます。詳細はコード内コメントを参照してください。

```

import Image from "next/image"; // default export の
Image コンポーネント

export function BookCover() {
  return (
    <Image
      src="/covers/book.jpg" //
      alt="書籍の表紙" // 代替テキスト（必須）
      width={200} // 元画像のアスペクト比計
      height={300} // 画像のレイアウト崩れを
      priority // LCP（最大コンテンツ描
    />
  );
}

```

public/ フォルダ: 画像や favicon などの静的ファイルは **public/** に置きます。URL では **/** から直接参照します（例: **public/logo.png** → **/logo.png**）。

6.6 next.config.js / next.config.ts（補足）

プロジェクトルートの **next.config.js**（または TypeScript 版 **next.config.ts**）は Next.js 全体の設定ファイルです。画像の許可ドメインや実験的機能の有効化などをここで指定します。

▼ このコードがやること（先に日本語で）：プロジェクト全体の設定ファイル **next.config.ts** の基本形を学びます。ここでは **next/image** で外部サイトの画像を表示してよいドメインを **remotePatterns** で許可する例を示します（許可しないと外部画像は表示されません）。設定オブジェクトを作り **export default** する、という決まった書き方を押さえてください。詳細はコード内コメントを参照してください。

```
// next.config.ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [
      // 設定ファイル本体
      // 型を取り込み
      // 設定オブジェクト
      // next/image の設定
      // 外部URLを許可するリ
      { protocol: "https", hostname: "example.com" },
    ],
  },
  // experimental: { ... }
  // 実験的機能はここで
  // ON にする
};

export default nextConfig;
// default export す
// ことで Next.js が読み込む
```

7. Server Actions

7.1 Server Actions とは

Server Actions は、Client Component から直接サーバー上の関数を呼び出せる 仕組みです。API ルートを別途作成する必要がなく、フォーム処理やデータの変更（作成・更新・削除）をシンプルに書けます。

従来の方法

Client Component
(フォーム)

fetch('/api/books')
→

API Route
(route.ts)

DB操作
→

データベース

Server Actions (新しい方法)

Client Component
(フォーム)

関数を直接呼び出し
→

Server Action
(サーバー上で実行)

DB操作
→

データベース

Server Actions は `"use server"` ディレクティブで宣言します。 `"use client"` がファイル全体を Client Component にするのと同じ要領で、 `"use server"` はファイルや関数を「サーバー上で動く関数」として印を付ける役割を持ちます。

7.2 フォーム処理での利用

基本的な Server Action

▼このコードがやること（先に日本語で）：フォーム送信を処理する関数（Server Action）を作ります。ファイル先頭の `"use server"` が「この中の関数はサーバー上で動く」という宣言で、これにより別途 API を作らなくてもフォームの保存処理が書けます。送信された値を `formData.get("title")` で取り出し、入力チェック（バリデーション）してから保存する流れを学べます。詳細はコード内コメントを参照してください。

```

// app/books/new/actions.ts                                     // Server Action 用の
専用ファイル                                                  // ファイル先頭の

"use server";                                                  //

`"use server"` で「この中の関数は全部サーバー実行」と宣言

// この関数はサーバー上でのみ実行される
// クライアントには関数の中身は送信されない

export async function createBook(formData: FormData) {         // 引数 `FormData` は
                                                                // <form action=

{createBook}> の送信内容が自動で渡される
    const title = formData.get("title") as string;             //
    `formData.get("title")` で name="title" の値を取り出す
                                                                // 戻り値は

`FormDataEntryValue | null` なので `as string` で型を絞り込む
    const author = formData.get("author") as string;           // 同じく著者
    const publishedYear = Number(formData.get("publishedYear")); // 数値に変換 (formData
の値は基本文字列)

    // バリデーション
    if (!title || !author) {                                    // 必須項目が空ならエラ
一を返す
        return {                                                // オブジェクトを返すと
呼び出し側で state として扱える
            error: "タイトルと著者は必須です",
        };
    }

    // データベースに保存 (例)                                  // 本番ではここで
Supabase や Prisma で INSERT
    // const book = await db.book.create({
    //   data: { title, author, publishedYear },
    // });

    console.log("書籍を追加:", { title, author, publishedYear }); // サーバーのコンソールに
出力 (ブラウザには出ない)

    // 成功したら書籍一覧にリダイレクト
    // redirect("/books");                                       // `redirect()` を呼
べば一覧ページに自動遷移
    return { success: true };                                    // 成功フラグを返す
}

```

Server Component のフォーム（JavaScript 不要）

▼ このコードがやること（先に日本語で）：先ほどの Server Action を使う「書籍追加フォーム」を Server Component として作ります。`<form action={createBook}>` のように `action` 属性に関数を直接渡せるのが Next.js の拡張で、JavaScript が動かない環境でも送信できる堅牢な作りになります（Progressive Enhancement）。各 `<input>` の `name` 属性が、サーバー側で値を取り出すときのキー名になる点が重要です。詳細はコード内コメントを参照してください。

```

// app/books/new/page.tsx (Server Component)
client" を書かないので Server

import { createBook } from "../actions";
actions.ts から Server Action を取り込む

export default function NewBookPage() {
  // 同期関数でOK (このページ自体は async データ取得をしない)
  return (
    <div>
      <h1>書籍を追加</h1>

      { /* action 属性に Server Action を渡す */ }
      { ... } >` に関数を渡せるのが Next.js の拡張
      { /* JavaScript が無効でも動作する (Progressive Enhancement) */ }
      // Progressive Enhancement: JSが動かない環境でもHTMLだけで動く設計
      <form action={createBook}>
        // 送信時に createBook がサーバーで実行される
        <div>
          <label htmlFor="title">タイトル</label>
          // `htmlFor` は HTML の `for` 属性のJSX版 (forはJSの予約語)
          <input
            type="text"
            id="title"
            // テキスト入力
            // label と紐付けるための id
            name="title"
            // ※ formData.get で取り出すキー名はこの name 属性
            required
            // 入力必須 (ブラウザの標準バリデーション)
          />
        </div>

        <div>
          <label htmlFor="author">著者</label>
          <input
            type="text"
            id="author"
            name="author"
            required
          />
        </div>

        <div>
          <label htmlFor="publishedYear">出版年</label>
          <input
            type="number"
            // 数値入力欄。上下ボタンが付く
            id="publishedYear"

```



```

        name="publishedYear"
      />
    </div>

    <button type="submit">追加する</button>           // submit ボタン
  </form>
</div>
);
}

```

画面にはこう表示される:「書籍を追加」という見出しの下に、タイトル、著者、出版年の入力フィールドと「追加する」ボタンが表示されます。各フィールドに入力して「追加する」ボタンを押すと、フォームのデータがサーバーに送信され、`createBook` 関数がサーバー上で実行されます。

Client Component のフォーム（ローディング状態の管理）

▼ このコードがやること（先に日本語で）: 同じフォームを、送信中の状態やエラー表示まで扱える Client Component 版に作り変えます。React 19 の `useActionState` フックを使うと、Server Action と連携しながら「現在の状態・加工済みの送信関数・送信中フラグ」の3つを受け取れます。送信中はボタンを「追加中...」にしてクリック不可にし、エラーがあれば画面に出す、という親切な UI が作れます。詳細はコード内コメントを参照してください。

```

// components/BookFormClient.tsx                                     // クライアント側でロ
ーディング状態を扱えるフォーム
"use client";                                                         // useActionState
は React のフックなので Client 必須

import { useActionState } from "react";                               // React 19 で追加され
た Server Action 連携用フック

                                                                    // 旧名は
useFormState (同じ用途)
import { createBook } from "@app/books/new/actions";                 // 別ファイルの Server
Action を取り込む

export function BookFormClient() {
  const [state, formAction, isPending] = useActionState(createBook, null);
                                                                    // 戻り値は [現在の
state, ラップ済み formAction, 送信中フラグ]
                                                                    // 第2引数の null は
state の初期値
                                                                    // formAction は
<form action={...}> に渡せるよう加工された関数

  return (
    <form action={formAction}>                                         // 元の createBook で
    はなく、ラップ済みの formAction を渡す
    <div>
      <label htmlFor="title">タイトル</label>
      <input type="text" id="title" name="title" required />         // name="title" が
formData のキーになる
    </div>

    <div>
      <label htmlFor="author">著者</label>
      <input type="text" id="author" name="author" required />
    </div>

    <div>
      <label htmlFor="publishedYear">出版年</label>
      <input type="number" id="publishedYear" name="publishedYear" />
    </div>

    {/* エラーメッセージの表示 */}
    {state?.error && (                                               // `?.` はオプショナ
ルチェイニング (state が null でも安全)
      <p className="error-message">{state.error}</p>                 // Server Action が返
した error メッセージを表示
    )}

    {/* 送信中はボタンを無効化 */}
    <button type="submit" disabled={isPending}>                       // `disabled=

```

```

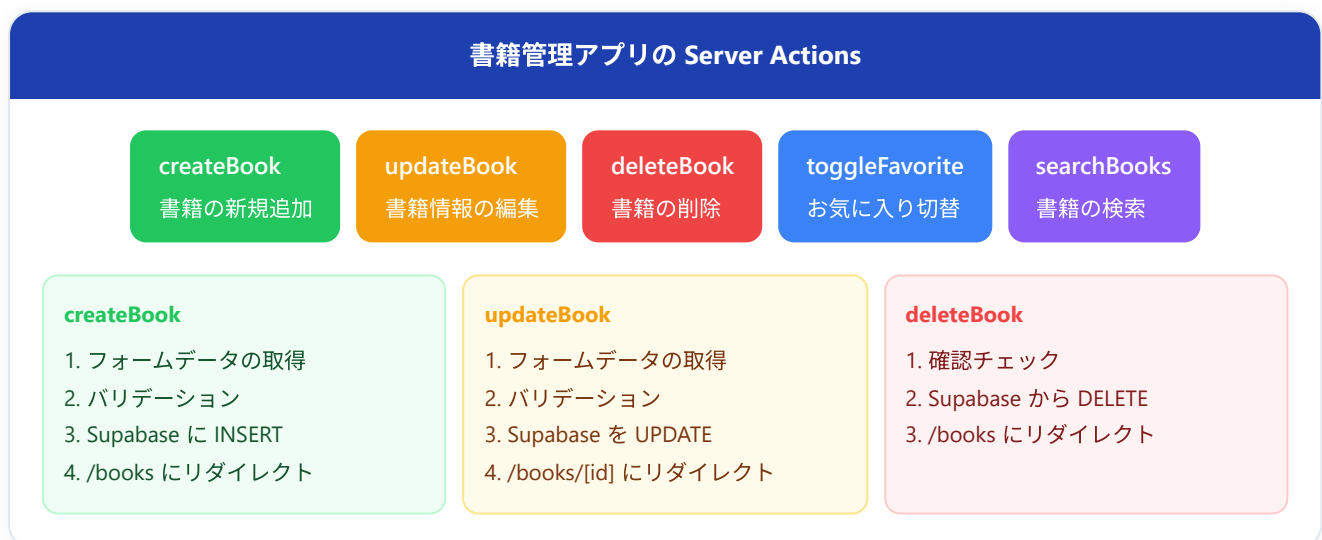
{isPending}` で送信中はクリック不可
      {isPending ? "追加中..." : "追加する"} // ボタンラベルも切り替
え
      </button>
    </form>
  );
}

```

画面にはこう表示される: フォームは前の例と同じ見た目ですが、「追加する」ボタンを押すと、ボタンのテキストが「追加中...」に変わり、ボタンがグレースアウトして再クリックできなくなります。バリデーションエラーがあれば、ボタンの上にエラーメッセージが赤字で表示されます。

7.3 書籍の追加/編集での利用予告

次章以降で、Server Actions を使って以下の機能を実装します:



予告: 第6章「Supabase との連携」で、実際のデータベースと接続してこれらの Server Actions を完成させます。

8. 環境変数

8.1 .env.local の設定方法

Next.js では、プロジェクトルートに `.env.local` ファイルを作成して環境変数を設定します。

▼ このコードがやること（先に日本語で）：APIキーやデータベース接続情報などの設定値を、コードに直接書かずに `.env.local` というファイルにまとめて管理します。 `KEY="値"` の形で1行に1つ書きます。変数名の先頭に `NEXT_PUBLIC_` を付けるとブラウザにも公開され、付けないとサーバー側だけで使える（＝機密情報を守る）という違いが最大のポイントです。詳細はコード内コメントを参照してください。

```
# .env.local (プロジェクトルートに作成)                                # `#` から始まる行はコメント。シェル/dotenv 共通の記法

# データベース接続情報 (サーバーサイドのみ)                          # NEXT_PUBLIC_ なしなのでブラウザには漏れない
DATABASE_URL="postgresql://user:password@localhost:5432/bookapp"      # `KEY="VALUE"` の形で1行1変数。`=` の前後にスペースを入れない

# Supabase 接続情報 (サーバーサイドのみ)                              # 機密キーは絶対
NEXT_PUBLIC_ を付けない
SUPABASE_URL="https://xxxxx.supabase.co"                             # Supabase プロジェクトのURL
SUPABASE_SERVICE_ROLE_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6I.. "        # サービスロールキー (DB全権限。絶対に公開してはいけない)

# Supabase 接続情報 (クライアントサイドでも使用可能)                # NEXT_PUBLIC_ プレフィックスでブラウザJSにも埋め込まれる
NEXT_PUBLIC_SUPABASE_URL="https://xxxxx.supabase.co"                 # 同じURLでも公開してよい想定なら NEXT_PUBLIC_ を付ける
NEXT_PUBLIC_SUPABASE_ANON_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6I.. "    # 匿名キー (Row Level Security で制限される前提なので公開OK)
```

重要: `.env.local` は `.gitignore` に含まれているため、Git にコミットされません。APIキーやパスワードなどの機密情報を安全に管理できます。

8.2 NEXT_PUBLIC_ プレフィックス

環境変数のアクセス範囲は、変数名のプレフィックスによって決まります。

NEXT_PUBLIC_あり	NEXT_PUBLIC_なし
NEXT_PUBLIC_SUPABASE_URL	SUPABASE_SERVICE_ROLE_KEY
✓ Server Component	✓ Server Component
✓ Client Component	✗ Client Component (undefined)
⚠ ブラウザのJSに含まれる	✓ ブラウザには絶対に送信されない

▼ このコードがやること（先に日本語で）：Server Component の中で環境変数を読み取る例です。サーバー側では `process.env.変数名` で、`NEXT_PUBLIC_` 付きでも無しでも、すべての環境変数にアクセスできます。機密キーもここでなら安全に使えます（ブラウザには送られない）。`console.log` の出力もブラウザではなくサーバーのターミナルに出る点を意識してください。詳細はコード内コメントを参照してください。

```
// Server Component での使用
// 環境変数にアクセスできる
// app/books/page.tsx

export default async function BooksPage() {
  // ✓ どちらもアクセス可能
  const url = process.env.SUPABASE_URL;
  // `process.env` は
  // Node.js 標準の環境変数オブジェクト
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;
  // 機密キー。Server
  // Component なので安全に取得可能
  const publicUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
  // 公開用キーも同様に取
  // れる

  console.log(url); // "https://xxxxx.supabase.co" // サーバーのコンソール
  // 出力（ブラウザDevToolsには出ない）
  console.log(serviceKey); // "eyJhbGciOi..." // ※ 本番ではこれをロ
  // グに出さないこと
  console.log(publicUrl); // "https://xxxxx.supabase.co"

  return <div>...</div>; // 省略表記
}
```

▼ このコードがやること（先に日本語で）：同じ `process.env` でも、Client Component（ブラウザ側）では `NEXT_PUBLIC_` 付きの変数しか読めない、という違いを確かめる例です。`NEXT_PUBLIC_` なしの機密キーを参照すると `undefined`（値なし）になり、これは「漏らさないための安全な仕様」です。どの変数がどちらで使えるかを区別できるようになるのが目標です。詳細はコード内コメントを参照してください。

```

// Client Component での使用 // ブラウザ側では
NEXT_PUBLIC_ 付きしか参照できない
// components/SomeClient.tsx
"use client"; // Client Component
化

export function SomeClient() {
  // ✅ NEXT_PUBLIC_ 付きはアクセス可能
  const publicUrl = process.env.NEXT_PUBLIC_SUPABASE_URL; // ビルド時に値がJSバンドルに埋め込まれる
  console.log(publicUrl); // "https://xxxxx.supabase.co" // ブラウザのコンソールに表示される

  // ❌ NEXT_PUBLIC_ なしは undefined
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY; // バンドルに含まれない
  =undefined // 機密情報が漏れない
  console.log(serviceKey); // undefined (安全!) // 設計になっている

  return <div>...</div>;
}

```

使い分けの原則:

環境変数の種類	プレフィックス	使用例
機密情報	なし	<code>SUPABASE_SERVICE_ROLE_KEY</code> 、 <code>DATABASE_URL</code>
公開しても安全な情報	<code>NEXT_PUBLIC_</code>	<code>NEXT_PUBLIC_SUPABASE_URL</code> 、 <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>

8.3 Supabase の接続情報の管理

書籍管理アプリでは、Supabase を使います。環境変数の設定例は以下の通りです。

▼ このコードがやること（先に日本語で）：書籍管理アプリで実際に使う Supabase（データベースサービス）の接続情報を `.env.local` に設定する具体例です。URL と匿名キーは公開してよいので `NEXT_PUBLIC_` を付け、全権限を持つサービスロールキーは絶対に公開しないため `NEXT_PUBLIC_` を付けません。この付け分けがそのままセキュリティの境界になります。詳細はコード内コメントを参照してください。

```

# .env.local                                     # 開発機ローカル用の環
境変数ファイル

# Supabase プロジェクト URL (公開OK)             # ブラウザにも露出して
OKな値
NEXT_PUBLIC_SUPABASE_URL="https://abcdefghijklm.supabase.co" # NEXT_PUBLIC_ 付き
でClient/Server 両方で使える

# Supabase 匿名キー (公開OK - Row Level Security で保護される)
NEXT_PUBLIC_SUPABASE_ANON_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." # 公開キー。RLS
で安全に絞られている前提

# Supabase サービスロールキー (絶対に公開してはいけない)
SUPABASE_SERVICE_ROLE_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." # NEXT_PUBLIC_ なし
=Server 限定

```

▼このコードがやること (先に日本語で) : ブラウザ側 (Client Component) から Supabase に接続するための「クライアント」を作る関数を定義します。公開してよい `NEXT_PUBLIC_` 付きの URL と匿名キーだけを使うのがポイントです。! (non-null assertion) は「この値は undefined ではないと TypeScript に約束する」記号で、設定漏れがあると実行時にエラーになります。詳細はコード内コメントを参照してください。

```

// lib/supabase/client.ts                       // ブラウザ側で動く
Supabase クライアント
// Client Component 用の Supabase クライアント

import { createBrowserClient } from "@supabase/ssr"; // `@supabase/ssr`
パッケージの import

export function createClient() {                  // クライアントを生成す
  関数 (毎回新しいインスタンスを作る)
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,        // `!` は TypeScript
    の non-null assertion (「undefinedではないと断言」)
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!    // 設定漏れがあるところ
    で undefined になり実行時エラーになる
  );
}

```

▼ このコードがやること（先に日本語で）：サーバー側（Server Component）から Supabase に接続するクライアントを作ります。クライアント版との違いは、ログイン状態を保つために Cookie を読み書きする仕組み（`cookies()` を使ったアダプタ）を渡している点です。`cookies()` が Promise を返すため関数を `async` にして `await` する必要があります。やや複雑ですが、いまは「サーバー用は Cookie 連携が要る」とだけ掴めば十分です。詳細はコード内コメントを参照してください。


```

// lib/supabase/server.ts // サーバー側で動く
Supabase クライアント
// Server Component 用の Supabase クライアント

import { createServerClient } from "@supabase/ssr"; // サーバー用ファクトリ
関数
import { cookies } from "next/headers"; // Next.js が提供す
る、リクエストの Cookie 読み書きAPI

export async function createClient() { // async にする必要が
あるのは cookies() が Promise を返すため
  const cookieStore = await cookies(); // 現在のリクエストの
Cookie ストアを取得

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!, // URL
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!, // 匿名キー
    {
      cookies: { // Cookie 入出力のアダ
        プタを渡す
        getAll() { // 全 Cookie を返す関
          数
            return cookieStore.getAll();
          },
          setAll(cookiesToSet) { // Supabase がセッショ
            ン更新時に呼ぶ関数
            try {
              cookiesToSet.forEach(({ name, value, options }) => // 配列を1つずつ処理
                cookieStore.set(name, value, options) // Next.js の cookie
                ストアに書き込み
              );
            } catch {
              // Server Component からの呼び出し時は // Server Component か
              ら cookie を書き込もうとすると例外が出るが
              // cookie のセットができないが、問題ない // 実害がないため握りつぶ
              してよい
            }
          },
        },
      },
    },
  );
}

```

次章で詳しく解説: Supabase のセットアップと実際の接続方法は、第5章「Supabase 入門」で詳しく扱います。ここでは環境変数の管理方法だけ理解しておいてください。

9. プロジェクト構成のベストプラクティス

9.1 ディレクトリ構成例

bookshelf-app/

app app/ (ルーティング・ページ)

layout.tsx

page.tsx

globals.css

books/

page.tsx

loading.tsx

error.tsx

layout.tsx

actions.ts

new/

page.tsx

[id]/

page.tsx

edit/

page.tsx

components components/ (再利用可能なUI)

ui/ (汎用UIパーツ)

Button.tsx | Input.tsx

books/ (書籍関連)

BookCard.tsx | BookForm.tsx | SearchBar.tsx

layout/ (レイアウト関連)

Header.tsx | Footer.tsx | Sidebar.tsx

lib lib/ (ユーティリティ)

supabase/

client.ts | server.ts

utils.ts

types types/ (型定義)

book.ts | user.ts

public public/ (静的ファイル)

images/

9.2 各ディレクトリの役割

ディレクトリ	役割	含まれるもの
<code>app/</code>	ルーティングとページ	page.tsx, layout.tsx, loading.tsx, error.tsx, Server Actions
<code>components/</code>	再利用可能な UI コンポーネント	ボタン、カード、フォーム、ヘッダー等
<code>lib/</code>	ビジネスロジックとユーティリティ	DB接続、ヘルパー関数、外部API呼び出し
<code>types/</code>	TypeScript 型定義	アプリ全体で使う型 (Book, User 等)
<code>public/</code>	静的ファイル	画像、フォント、favicon 等

9.3 書籍管理アプリのファイル構成

以下は、書籍管理アプリの最終的なファイル構成の全体像です。

▼ このコードがやること（先に日本語で）：アプリ全体で使い回す「書籍データの形」を TypeScript の型として1か所にまとめて定義します。型を決めておくと、`book.title` のように書いたときにエディタが補完してくれたり、入力ミスを事前に警告してくれます。`string | null`（文字列か空のどちらか）や `?`（省略可能）といった、型を柔軟に表す書き方も学べます。詳細はコード内コメントを参照してください。

```

// types/book.ts // 型定義の置き場
// アプリ全体で使う書籍の型定義

export type Book = { // 名前付き export。
  id: string; // データベース上の主
  // キー (UUID または文字列ID)
  title: string; // 書籍タイトル
  author: string; // 著者名
  published_year: number; // 出版年。スネークケ
  // ースは Supabase の列名と合わせるため
  description: string | null; // 説明文 (null 許
  // 容)。`string | null` は型のユニオン
  cover_image_url: string | null; // 表紙画像のURL (任
  // 意)
  is_favorite: boolean; // お気に入りフラグ
  created_at: string; // 作成日時 (ISO 8601
  // 文字列)
  updated_at: string; // 更新日時
};

// フォーム送信時に使う型 (id や日時は不要)
export type BookFormData = { // 新規追加・編集フォ
  // ム専用の入力データ型
  title: string;
  author: string;
  published_year: number;
  description?: string; // `?` で「省略可能」
  // のオプションプロパティ
};

```

▼ このコードがやること（先に日本語で）：アプリのあちこちで使う「便利な小さな関数（ユーティリティ）」を1か所にまとめます。日付文字列を「2024年1月15日」の形に整える `formatDate` と、複数のクラス名を空白区切りでつなげる `cn` の2つを定義します。...`classes`（レストパラメータ）で引数を何個でも受け取る書き方や、`filter(Boolean)` で空の値を除く書き方が学べます。詳細はコード内コメントを参照してください。

```

// lib/utils.ts // 汎用ヘルパー置き場
// 汎用ユーティリティ関数

/**
 * 日付文字列を日本語のフォーマットに変換する
 * @example formatDate("2024-01-15") → "2024年1月15日" // `@example` は
JSDoc の例示タグ。IDEで吹き出しに表示される
 */
export function formatDate(dateString: string): string { // 引数も戻り値も
string と明示
    const date = new Date(dateString); // 文字列を Date オブ
ジェクトに変換
    return date.toLocaleDateString("ja-JP", { // ロケール指定で日本
語フォーマット
        year: "numeric", // 年は数字 (2024)
        month: "long", // 月は「1月」のような
長い表記
        day: "numeric", // 日は数字
    });
}

/**
 * クラス名を結合する (falsy な値は除外)
 * @example cn("base", isActive && "active", "extra") → "base active extra"
 */
export function cn(...classes: (string | false | undefined | null)[]): string {
// `...classes` はレ
ストパラメータ。可変長の引数を配列として受け取る
// 型は「string か
false か undefined か null の配列」
    return classes.filter(Boolean).join(" "); //
`filter(Boolean)` で false/undefined/null/"" を除外
// `join(" ")` で半
角スペース区切りに連結
}

```

▼ このコードがやること（先に日本語で）：色や種類を切り替えられる、使い回し可能な汎用ボタン部品を作ります。variant（primary/secondary/danger）や disabledなどを props で受け取り、種類に応じて CSS クラスを組み立てます。variant = "primary" のように引数にデフォルト値を持たせる書き方や、props の形を型（ButtonProps）で定義する書き方が学べます。詳細はコード内コメントを参照してください。

```

// components/ui/Button.tsx
コンポーネント
// 汎用ボタンコンポーネント

type ButtonProps = {
  children: React.ReactNode;
  (テキストやアイコン)
  variant?: "primary" | "secondary" | "danger";
  // props の型定義
  // ボタン内の表示内容
  // ボタンの種類。`?`
  disabled?: boolean;
  // 無効化フラグ
  type?: "button" | "submit";
  // HTML の button 要
  // クリック時のハンド
  // 素の type
  onClick?: () => void;
  // ラ (任意)
};

export function Button({
  // 分割代入で props
  // 取り出す
  children,
  variant = "primary",
  // デフォルト値の指
  // 定。指定がなければ "primary"
  disabled = false,
  type = "button",
  // デフォルトは
  "button" (submit を防ぐ)
  onClick,
}: ButtonProps) {
  // 全ボタン共通のクラ
  // ス
  const baseClass = "btn";
  // 種類別のクラス (例:
  // "btn-primary")

  const variantClass = `btn-${variant}`;

  return (
    <button
      type={type}
      // HTML 属性に渡す
      className={` ${baseClass} ${variantClass}`}
      // テンプレートリテラ
      // ルで2つのクラスを連結
      disabled={disabled}
      onClick={onClick}
    >
      {children}
      // ボタンの中身を表示
    </button>
  );
}

```

パスエイリアスの設定

`@/` というパスエイリアスを使うと、ディレクトリの深さに関係なく、プロジェクトルートからの絶対パスで import できます。Next.js のプロジェクト作成時に自動で設定されます。

```
// ❌ 相対パスでの import (ディレクトリが深いと地獄)
import { BookCard } from "../../components/books/BookCard"; // `../` の数を間違えると即エラー

import { formatDate } from "../../lib/utils"; // ファイルを移動するたびに書き直しが必要

// ✅ パスエイリアスでの import (常にわかりやすい)
import { BookCard } from "@components/books/BookCard"; // `@/` がプロジェクトルートを指す
import { formatDate } from "@lib/utils"; // どんなに深い階層からでも同じ書き方でOK
```

この設定は `tsconfig.json` (TypeScript のコンパイル設定ファイル) に記述されています:

```
{
  "compilerOptions": { // TypeScript コンパ
    イラのオプション
    "paths": { // パスエイリアスのマ
      ッピング
      "@/*": ["./*"] // `@/` を「プロジェ
      クトルートからの相対パス」に展開
    }
  }
}
```

10. よくあるエラーと対処法

エラー1: "useState" は Server Component で使用できない

```
Error: useState only works in Client Components. Add the "use client" directive
at the top of the file to use it.
```

原因: Server Component (デフォルト) で `useState` や `useEffect` を使おうとしている。

対処法: ファイルの先頭に `"use client"` を追加する。

```

// ❌ エラーになる // "use client" がない
// での Server Component 扱い
import { useState } from "react"; // React のフックを取り込み

export default function Counter() {
  const [count, setCount] = useState(0); // ← ここでエラー // Server Component では
  // useState を呼べない
  return <button onClick={() => setCount(count + 1)}>{count}</button>; // onClick も
}
// Server Component では使えない

// ✅ 修正版
"use client"; // ← これを追加 // ファイル先頭のディレ
// クティブで Client Component 化

import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0); // OK: Client では
  // useState が使える
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}

```

エラー2: "async/await" は Client Component で使用できない

```
Error: async/await is not yet supported in Client Components, only Server Components.
```

原因: `"use client"` を付けたコンポーネントで `async` 関数としてエクスポートしている。

対処法: データ取得を Server Component に移動するか、`useEffect` を使う。


```

// ✖ エラーになる
"use client"; // Client Component
なのに

export default async function BooksPage() { // 関数本体に async
  を付けている
  const books = await fetch("/api/books"); // ← async は Client Component で不可
  // App Router の
  Client では関数を async にできない
  return <div>...</div>;
}

// ✔ 修正版 1: Server Component にする ("use client" を削除)
export default async function BooksPage() { // "use client" がな
  ければ Server Component
  const books = await fetch("https://api.example.com/books"); // Server なら async
  でも OK
  return <div>...</div>;
}

// ✔ 修正版 2: Client Component のまま useEffect を使う
"use client";

import { useState, useEffect } from "react";

export default function BooksPage() { // 関数本体は同期のま
  まにする
  const [books, setBooks] = useState([]); // 結果を state に保
  持

  useEffect(() => { // マウント後にデータ
    取得
    fetch("/api/books")
      .then((res) => res.json()) // Promise チェーン
      .then(setBooks); // 取れた配列を state
    に格納
  }, []); // 依存配列が空なので
  初回マウント時に1度だけ実行

  return <div>...</div>;
}

```

エラー3: "next/router" と "next/navigation" の混同

```
Error: NextRouter was not mounted.
```

原因: App Router で `next/router` を import している。App Router では `next/navigation` を使う。

```
// ❌ App Router では使えない
import { useRouter } from "next/router";           // 旧 Pages Router
用の useRouter

// ✅ App Router ではこちらを使う
import { useRouter } from "next/navigation";       // 新しい App Router
用の push/replace/back/refresh などを持つ
```

エラー4: metadata と "use client" の共存

Error: You are attempting to export "metadata" from a component marked with "use client", which is unsupported.

原因: `"use client"` のファイルで `metadata` をエクスポートしている。メタデータは Server Component でのみエクスポート可能。

```
// ❌ エラーになる
"use client";                                     // Client Component
なのに

export const metadata = { title: "書籍一覧" }; // ← Client Component では不可
                                                // metadata は
Server Component の機能なので Client では宣言できない

// ✅ 修正版: metadata は Server Component (page.tsx や layout.tsx) に置く
// app/books/page.tsx (Server Component)
export const metadata = { title: "書籍一覧" }; // この行でブラウザタ
ブのタイトルが「書籍一覧」になる

export default function BooksPage() {
  return <div>...</div>;
}
```

補足: `generateMetadata` : 動的にメタデータを決めたいとき（例: 書籍IDから書籍タイトルを取って `<title>` に入れる）は、`generateMetadata` という名前で async 関数を export します。引数として `params` を受け取れます。同様に「ビルド時に動的セグメントの値を事前列挙したい」場合は `generateStaticParams` を export します。

エラー5: 環境変数が undefined になる

原因: Client Component で `NEXT_PUBLIC_` プレフィックスのない環境変数を使っている。

```
// ❌ Client Component では undefined になる
"use client";

const apiKey = process.env.API_SECRET_KEY; // undefined           // NEXT_PUBLIC_ なし
// 環境変数はブラウザバンドルに含まれない

// ✅ NEXT_PUBLIC_ を付ける (ただし機密情報には使わない!)
const publicUrl = process.env.NEXT_PUBLIC_API_URL; // 値が取得できる // 公開してよい値だけ
// に NEXT_PUBLIC_ を付ける
```

注意: 機密情報に `NEXT_PUBLIC_` を付けてはいけません。Server Action や API Route 経由でアクセスしてください。

エラー6: "Text content does not match server-rendered HTML" (Hydration エラー)

```
Warning: Text content did not match. Server: "2024年3月15日" Client: "3/15/2024"
```

原因: サーバーとクライアントで異なる出力が生成されている。日時のフォーマットやランダム値でよく発生する。

```
// ❌ サーバーとクライアントで結果が異なる可能性がある
export default function DateDisplay() {
  return <p>{new Date().toLocaleString()}</p>;
  // サーバーとブラウザ
  // → Hydration エラ
  // ーになる
}

// ✅ 修正版 1: suppressHydrationWarning を使う
export default function DateDisplay() {
  return <p suppressHydrationWarning>{new Date().toLocaleString()}</p>;
  // React の
  // Hydration 不一致警告を抑制する属性
  // 中身が動的でもよい
  // 場合の応急処置
}

// ✅ 修正版 2: Client Component にして useEffect で設定
"use client";
// Client 化

import { useState, useEffect } from "react";

export default function DateDisplay() {
  const [dateStr, setDateStr] = useState("");
  // 初期値は空文字（サ
  // ーバーでも同じ値）

  useEffect(() => {
    // ハイドレーション後
    // = ブラウザ側でのみ走る
    // ここで日付を設定 →
    // 再描画
    setDateStr(new Date().toLocaleString("ja-JP"));
    // 初回1回だけ
  }, []);

  return <p>{dateStr}</p>;
  // サーバー描画時は
  // 空、ブラウザで時刻に置き換わる
}

```

エラー7: fetch でのキャッシュに関する警告

Warning: fetch for "https://..." on "/" used "no-store" and should not be called inside a layout or template.

対処法: `layout.tsx` 内では `cache: "no-store"` を避け、`page.tsx` で使う。

エラーの調査方法（一般的なアドバイス）

1. **ターミナルのエラーメッセージを読む:** Next.js のエラーメッセージは非常に詳しく、多くの場合、解決策まで提示してくれます。
2. **ブラウザのコンソールを確認する:** 開発者ツール（F12）の Console タブにもエラーが表示されます。
3. **Server Component と Client Component の境界を確認する:** 多くのエラーは、この2つの境界での誤りが原因です。
4. **Next.js の公式ドキュメントを参照する:** <https://nextjs.org/docs> には詳細な説明とトラブルシューティングがあります。

まとめ

この章では Next.js の基礎を広く学びました。

トピック	学んだこと
Next.js とは	React のフルスタックフレームワーク。SSR/SSG/CSR に対応
App Router	ファイルベースルーティング。page.tsx, layout.tsx 等の特殊ファイル
Server/Client Components	デフォルトは Server。インタラクティブ性が必要なら "use client"
レイアウト	ルートレイアウトとネストされたレイアウトで共通UIを効率的に管理
ナビゲーション	Link コンポーネント（宣言的）と useRouter フック（命令的）
データフェッチ	Server Component なら async/await で直接。Client は useEffect
Server Actions	"use server" でサーバー処理を直接呼び出し。フォーム処理に最適
環境変数	NEXT_PUBLIC_ の有無でアクセス範囲が変わる
プロジェクト構成	app/, components/, lib/, types/ の4層構造

次の章では: Supabase のセットアップを行い、実際のデータベースと接続します。書籍管理アプリのバックエンドを構築し、この章で学んだ Server Components や Server Actions を使ってデータの読み書きを実装していきます。

確認問題

以下の問いに答えて、理解度を確認してみましょう。

1. **Server Component と Client Component の違い** を3つ挙げてください。
2. 以下のコンポーネントは Server / Client のどちらにすべきですか？ - データベースから書籍一覧を取得して表示するコンポーネント - 検索バー（ユーザーの入力をリアルタイムに反映） - 書籍カードの静的な表示（クリックイベントなし）

3. `NEXT_PUBLIC_API_KEY` と `API_SECRET_KEY` はそれぞれどこで使用できますか？
4. `layout.tsx` と `page.tsx` の違いは何ですか？
5. Server Actions を使う利点を2つ挙げてください。

▶ 解答を見る